

STUDY GUIDE · HELM 4 · CHARTS, VALUES, RELEASES

Helm From Zero

Complete Hands-On Helm Learning Lab

From 1,374 lines of copied Kubernetes YAML to one chart with three short values files. The YAML problem, what Helm is, charts and chart versions, values and precedence, templates and helpers, rendering and linting, releases, upgrades and rollbacks, multiple environments, repositories and OCI, dependencies, hooks and tests, ConfigMaps and Secrets, security, reviewing changes, best practices, a production-style chart; plus the cheat sheet, the troubleshooting method, the capstone, a glossary and 25 interview questions.

Sufyan Ahmad · DevOps Engineer

<https://github.com/sufyanahmadkamboh/sufyan-devops-helm-from-zero>

Contents

[Study material](#)

[Level 1 · The Kubernetes YAML problem](#)

[Level 2 · What is Helm?](#)

[Charts and Chart.yaml](#)

[Values](#)

[Templates, helpers and labels](#)

[Rendering and linting](#)

[Releases: install, status, history, uninstall](#)

[Upgrades and rollbacks](#)

[Multiple environments](#)

[Repositories and OCI registries](#)

[Chart dependencies](#)

[Hooks, tests and NOTES](#)

[ConfigMaps and Secrets in charts](#)

[Level 17 · Helm security](#)

[Reviewing changes before an upgrade \(Helm diff\)](#)

[Helm best practices](#)

[Level 19 · A production-style chart](#)

[Helm cheat sheet](#)

[Troubleshooting labs](#)

[Lab 16 · Capstone: from raw YAML to a Helm-managed platform](#)

[Glossary](#)

[Interview questions](#)

Each concept page answers the same ten questions: what it is, why we need it, how it works, what problem it solves, how to use it, which command, which output, what can go wrong, how to troubleshoot it, and where it is used in real DevOps work. The tested hands-on labs (00–16), the twelve troubleshooting scenarios, the challenges and the 24-chapter guided course are in the repository.

Study material

Everything you need to revise after (or alongside) the hands-on labs.

File	What it is	Use it when
study-guide.pdf	Level 1, the 16 concept pages, the cheat sheet, the troubleshooting method, the capstone, the glossary and the interview questions as one printable PDF (answers expanded, with a table of contents; built by <code>study/tools/build_pdf.py</code>)	you want to read offline, print, or annotate
glossary.md	every term used in the course, A–Z, in one or two plain sentences, with a link to where it is explained	a word in a lesson is unclear
interview-questions.md	25 questions with model answers, from chart versions to server-side apply conflicts	before an interview, or to test yourself
../tutorial/	the 24-chapter guided course through the repository	you want to be taught, step by step

How to revise

1. Work through the [tutorial](#); it sends you to the labs and docs in the right order.
2. After each chapter, look up every glossary term you could not explain to a colleague.
3. At the end, answer the interview questions **out loud**, then compare with the model answers. Explaining is a different skill from knowing; it needs practice.
4. Finish with the [capstone](#), the [challenges](#) and the [final review](#).

Rebuild the PDF

```
pip install markdown
python study/tools/build_pdf.py      (uses a headless Chrome or Edge to print the PDF)
```

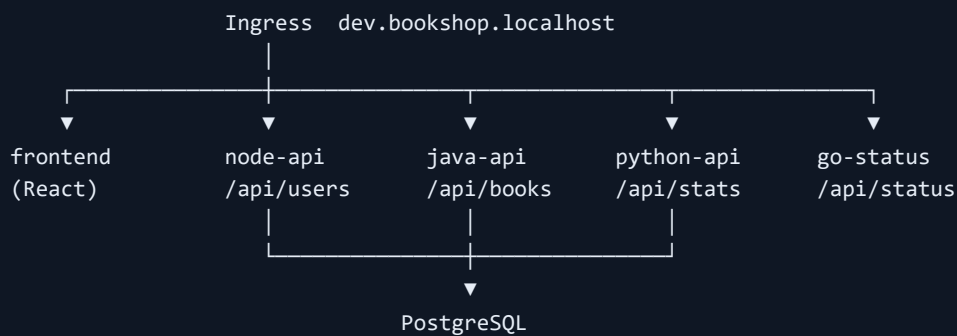
Level 1 · The Kubernetes YAML problem

Time: 20 minutes. Needs the cluster from [lab 00](#). No Helm in this lesson, on purpose.

Before Helm, let's see the problem. This folder is the Bookshop platform (from the [multi-stack project](#)), deployed the way you already know: plain Kubernetes YAML and `kubectl apply`. Three environments, three folders:

```
environments/
├── dev/
│   ├── namespace.yaml
│   ├── configmap.yaml
│   ├── secret.yaml
│   ├── database.yaml
│   ├── deployments.yaml
│   ├── services.yaml
│   └── ingress.yaml
├── staging/
│   ├── namespace.yaml
│   ├── configmap.yaml
│   ├── secret.yaml
│   ├── database.yaml
│   ├── deployments.yaml
│   ├── services.yaml
│   └── ingress.yaml
└── prod/
    ├── namespace.yaml
    ├── configmap.yaml
    ├── secret.yaml
    ├── database.yaml
    ├── deployments.yaml
    ├── services.yaml
    └── ingress.yaml
```

(StatefulSet + 5 services)



Step 1 · Deploy dev, the way you know

The namespace first (the other files create objects inside it), then the whole folder:

```
kubectl apply -f environments/dev/namespace.yaml
kubectl apply -f environments/dev/
```

```
...
secret/db-credentials created
service/frontend created
service/node-api created
service/python-api created
service/go-status created
service/java-api created
```

```
for d in frontend node-api java-api python-api go-status; do
  kubectl -n bookshop-dev rollout status deployment/$d --timeout=300s
done
```

Verify, as you would for any deployment:

```
kubectl -n bookshop-dev get deployments
kubectl -n bookshop-dev get pods
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
frontend	1/1	1	1	24s
go-status	1/1	1	1	24s
java-api	1/1	1	1	24s
node-api	1/1	1	1	24s
python-api	1/1	1	1	24s

NAME	READY	STATUS	RESTARTS	AGE
frontend-56687b87df-fx5lb	1/1	Running	0	24s
go-status-65f4db8dbf-ds7z2	1/1	Running	0	24s
java-api-7849f88465-b2sqb	1/1	Running	0	24s
node-api-547688d4-j9rt5	1/1	Running	0	24s
postgres-0	1/1	Running	0	24s
python-api-755bf48f77-2q6mf	1/1	Running	0	24s

```
kubectl -n bookshop-dev get services
kubectl -n bookshop-dev get ingress
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
frontend	ClusterIP	10.96.3.155	<none>	8080/TCP	24s
go-status	ClusterIP	10.96.105.96	<none>	8080/TCP	24s
java-api	ClusterIP	10.96.127.78	<none>	8080/TCP	24s
node-api	ClusterIP	10.96.67.10	<none>	3000/TCP	24s
postgres	ClusterIP	None	<none>	5432/TCP	24s
python-api	ClusterIP	10.96.195.177	<none>	8000/TCP	24s

NAME	CLASS	HOSTS	ADDRESS	PORTS	AGE
bookshop	traefik	dev.bookshop.localhost		80	24s

```
curl -s http://dev.bookshop.localhost:8080/config.js; echo
curl -s http://dev.bookshop.localhost:8080/api/users | head -c 120; echo
```

```
window.APP_CONFIG = { ADMIN_URL: "", APP_ENV: "dev" };

[{"id":1,"name":"Ada Lovelace","email":"ada@example.com","created_at":"2026-10-04T23:27:03.935Z"},
{"id":2,"name":"Grace
```

Open <http://dev.bookshop.localhost:8080/> in a browser: the Bookshop, with `dev` in its footer.

What works: everything. Plain YAML is explicit, reviewable, and `kubectl apply` is all you need. For one application in one environment this is a perfectly good way to work.

Step 2 · Now imagine staging and production

Staging and production run the same application. Same Deployments, same Services, same probes, same security settings. How much of each folder is actually different?

```
wc -l environments/dev/*.yaml | tail -1
wc -l environments/staging/*.yaml | tail -1
wc -l environments/prod/*.yaml | tail -1
diff -r environments/dev environments/staging | grep -c '^[<>]'
```

```
458 total
458 total
458 total
74
```

About 460 lines per environment; the `diff` counts the lines that differ between dev and staging (each changed line counts twice: once `<`, once `>`). Everything else is a copy. What differs?

```
diff environments/dev/deployments.yaml environments/staging/deployments.yaml || true
```

```
1c1
< # Bookshop, environment: dev. Deployed WITHOUT Helm: kubectl apply -f environments/dev/namespace.yaml
-f environments/dev/
---
> # Bookshop, environment: staging. Deployed WITHOUT Helm: kubectl apply -f
environments/staging/namespace.yaml -f environments/staging/
6c6
< namespace: bookshop-dev
---
> namespace: bookshop-staging
9c9
< replicas: 1
---
> replicas: 2
31c31
< value: dev
---
> value: staging
37c37
< value: http://node-api.bookshop-dev.svc.cluster.local:3000
---
> value: http://node-api.bookshop-staging.svc.cluster.local:3000
39c39
< value: http://java-api.bookshop-dev.svc.cluster.local:8080
---
> value: http://java-api.bookshop-staging.svc.cluster.local:8080
41c41
< value: http://python-api.bookshop-dev.svc.cluster.local:8000
---
> value: http://python-api.bookshop-staging.svc.cluster.local:8000
43c43
< value: http://go-status.bookshop-dev.svc.cluster.local:8080
---
> value: http://go-status.bookshop-staging.svc.cluster.local:8080
63c63
< namespace: bookshop-dev
---
> namespace: bookshop-staging
66c66
< replicas: 1
---
> replicas: 2
...
```

The namespace, the replica counts, the `APP_ENV` value and the namespace inside every service URL. In `ingress.yaml` the host name; in `database.yaml` the volume size; in `secret.yaml` the password.

Step 3 · A routine change

The node-api team released version `1.1.0`. Where do you change the image tag?

```
grep -rn --include="*.yaml" "bookshop-node-api:" environments/ | sed 's/ *#.*//'
```

```
environments/dev/deployments.yaml:79:          image: ghcr.io/sufyanahmadkamboh/bookshop-node-api:1.0.0
environments/prod/deployments.yaml:79:          image: ghcr.io/sufyanahmadkamboh/bookshop-node-api:1.0.0
environments/staging/deployments.yaml:79:          image: ghcr.io/sufyanahmadkamboh/bookshop-node-
api:1.0.0
```

Three files, one per environment, and you must not forget one. The same is true for every probe setting, every resource limit, every new environment variable: **one change = N edits**, where N is the number of environments. Add a fourth environment (a load-test environment, a per-customer copy) and you copy 460 more lines.

Step 4 · And the secret

```
grep -n -A1 "stringData" environments/dev/secret.yaml
```

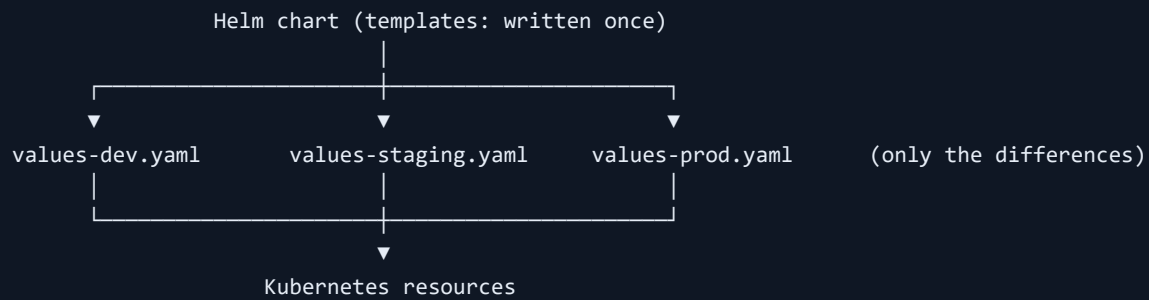
```
10:stringData:
11- DB_PASSWORD: example-only-dev-password
```

To make the folder applicable as-is, the password is written in a file. In a Git repository, that file is readable by everyone who can read the repository, forever (Git keeps history). Base64 in a Secret is encoding, not encryption.

What becomes difficult

	Without a tool
Repetition	Every environment is a full copy; 90 % of the lines are identical
Changes	One change means N edits; a forgotten copy means environments silently drift apart
Differences	Hidden inside 460 lines: you find them only with <code>diff</code>
Versions	Which version is running in staging? You read the YAML, or the cluster
History	<code>kubectl apply</code> keeps no history of what you applied, and there is no "undo the whole application"
Removal	You must remember every object you created
Secrets	Tempting to commit them next to the rest
Sharing	Another team wants your app? They copy your files and edit them too

What we want instead: **one** description of the application, with the differences between environments as a small list of values, a record of every deployment, and a one-command rollback. That is Helm:



The chart that replaces these 1,374 lines is [charts/bookshop](#): about 600 lines of templates and defaults (with many comments), written once, plus a values file per environment: 3 lines for dev, 9 for staging, 32 for production (which adds autoscaling and disruption budgets the raw YAML doesn't even have). You will build up to it, starting in [lab 01](#).

Cleanup

⚠️ DESTRUCTIVE COMMAND · deletes the namespace bookshop-dev and everything in it, including the database volume.

```
kubectl delete namespace bookshop-dev
```

Even the cleanup shows the problem: it works because everything happened to be in one namespace. Objects outside it (cluster-wide ones) you would have to remember yourself. Helm remembers what it created.

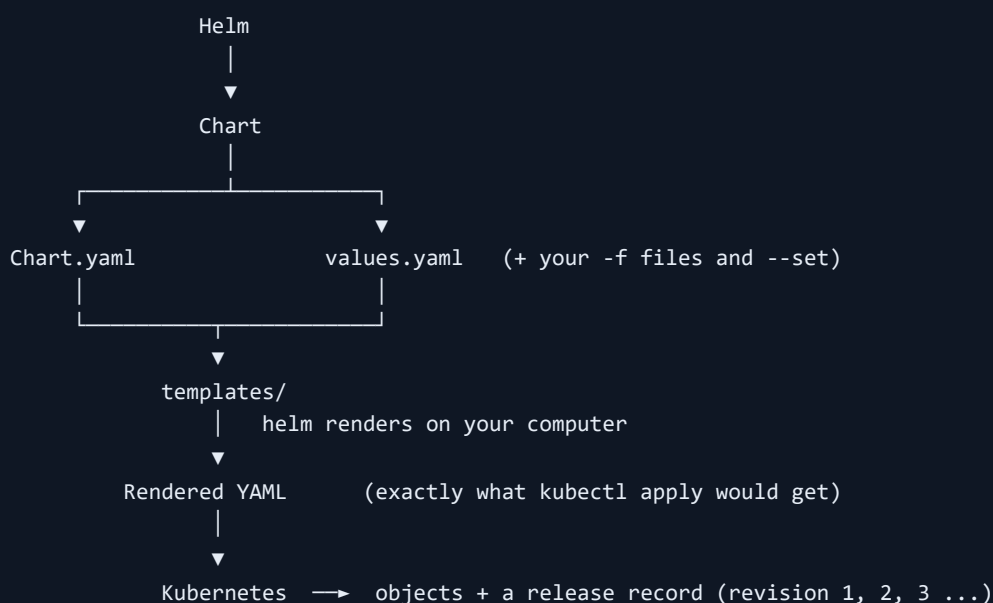
Next, Level 2: [What is Helm?](#)

Level 2 · What is Helm?

Time: 15 minutes of reading. You have just seen [the YAML problem](#).

Helm is the **package manager for Kubernetes**. It does three jobs:

1. **Packaging:** an application's Kubernetes YAML becomes a **chart**: a folder (or a `.tgz` file) of templates and default settings, with a name and a version.
2. **Templating:** at install time, Helm combines the templates with **values** (your settings) and produces plain Kubernetes YAML.
3. **Release management:** every install of a chart is a **release**, with a name, a numbered history of **revisions**, a status, an upgrade path and a rollback.



The vocabulary

Term	What it is	In this repository
Chart	A package: <code>Chart.yaml</code> + <code>values.yaml</code> + <code>templates/</code>	charts/demo-app
Values	The settings a chart accepts, with defaults in <code>values.yaml</code>	<code>replicaCount</code> , <code>image.tag</code> , <code>environment</code>
Template	A Kubernetes manifest with placeholders: <code>replicas: {{ .Values.replicaCount }}</code>	deployment.yaml
Rendered manifest	A template after Helm filled it in: plain YAML	<code>helm template ...</code> output
Release	One installed instance of a chart, with a name, in a namespace	<code>demo</code> in <code>demo-dev</code>

Term	What it is	In this repository
Revision	One version of a release; every install, upgrade, rollback adds one	<code>helm history demo</code>
Repository	A server that hosts packaged charts, with an index	<code>https://traefik.github.io/charts</code>
Chart version	The version of the package (<code>version:</code> in Chart.yaml)	<code>1.0.0</code>
App version	The version of the application it deploys (<code>appVersion:</code>)	<code>1.0.0</code> = image tag

One chart, many releases: the same `demo-app` chart is installed three times in lab 10, as three releases (dev, staging, prod), each with its own values and its own history.

Chart version vs application version

The distinction that confuses most beginners. Two different things change independently:

```
Chart.yaml
version: 1.2.0      ← the PACKAGING. Bump it whenever anything in the chart changes:
                    a template, a default value, a new option. SemVer: 1.2.0 → 1.2.1 (fix), 1.3.0
                    (feature).
appVersion: "2.0.0" ← the APPLICATION inside. The image tag deployed by default.
```

- A new template option (say, a PodDisruptionBudget) with the same application: chart `1.2.0` → `1.3.0`, app stays `2.0.0`.
- The developers release `2.1.0` and nothing else changes: app `2.0.0` → `2.1.0`, and the chart version still has to change (`1.3.0` → `1.3.1`), because the package now has different contents. A chart version is never reused for different contents.

`helm list` shows both: the `CHART` column (`bookshop-1.2.0`) and the `APP VERSION` column (`1.0.0`).

The 10 questions

1. What is it? A CLI that packages Kubernetes applications as charts, renders them with values, and manages installed instances (releases) with history.

2. Why do we need it? Because real applications run in several environments and are installed by several teams; copying YAML per environment does not scale (Level 1: 1,374 lines in three folders that are almost identical copies).

3. How does it work? The CLI loads the chart, merges values (defaults ← files ← `--set`), renders the Go templates into YAML, sends the objects to the Kubernetes API, and stores a release record (the rendered manifest and the values of that revision) as a Secret in the release's namespace. No component runs in the cluster.

4. What problem does it solve? Duplication (one template, many environments), drift (each environment is the chart + a short values file), history (revisions and rollback), cleanup (uninstall removes what the release created), distribution (repositories, versions) and reuse (installing other teams' charts).

5. How do I use it? Create or download a chart, choose values, `helm install`; later `helm upgrade`, `helm rollback`, `helm uninstall`. Labs 02–16 do all of it.

6. What command should I run? `helm template` to see what would be applied, `helm upgrade --install` to apply it, `helm history` and `helm rollback` when something goes wrong. The [cheat sheet](#) has the rest.

7. What output should I expect? Plain Kubernetes YAML from `helm template`; from `helm install`, a summary (`STATUS: deployed` , `REVISION: 1`) and the chart's NOTES.

8. What can go wrong? Template errors (caught by `helm lint` / `helm template`), wrong values (the YAML is valid but wrong), objects that fail in the cluster (bad image, failing probes): Helm only knows what Kubernetes reports. [Troubleshooting](#) has 12 cases.

9. How do I troubleshoot it? Render first (`helm template`), then compare what Helm applied (`helm get manifest` , `helm get values`) with what runs (`kubectl get/describe/logs`), then use the release history.

10. Where is it used in real DevOps work? Installing infrastructure (ingress controllers, cert-manager, Prometheus: almost every one ships as a Helm chart), packaging your company's services with one chart per service or one shared chart, deploying from CI/CD (`helm upgrade --install` in a pipeline) or from GitOps tools (Argo CD and Flux render Helm charts).

What Helm is not

- **Not a replacement for Kubernetes knowledge.** The output is Deployments, Services and Ingresses; when a Pod crashes, you debug it with `kubectl`, as always.
- **Not a secret manager.** A password in `values.yaml` is a password in Git ([security](#)).
- **Not the only option.** Customize patches plain YAML without templates (built into `kubectl`); it is a good fit for small variations of your own manifests. Helm adds packaging, versioning, distribution and release history. Many teams use both.

Next, Level 3: [lab 01 · Install Helm](#).

Charts and Chart.yaml

Hands-on: [lab 02](#), [lab 03](#).

1 • What is it?

A **chart** is a folder (or a `.tgz` of that folder) that describes a Kubernetes application:

```
demo-app/
├─ Chart.yaml      identity: name, versions, description, dependencies
├─ values.yaml     the settings, with defaults
├─ values-*.yaml   (convention) one file per environment
├─ charts/         dependencies, downloaded
├─ templates/      manifests with placeholders, helpers, NOTES.txt, tests/
├─ values.schema.json (optional) rules the values must follow
├─ README.md       (optional) shown by helm show readme
└─ .helmignore     files left out of the package
```

`Chart.yaml` is its identity card:

```
apiVersion: v2          # chart format (Helm 3 and 4), not a Kubernetes API version
name: demo-app          # the chart's name: package name, default resource names, the
                        app.kubernetes.io/name label
description: The Bookshop frontend ...
type: application       # or "library": helpers only, cannot be installed
version: 1.0.0          # the CHART's version (SemVer)
appVersion: "1.0.0"     # the APPLICATION's version: the default image tag
```

2 • Why do we need it?

To ship an application as one versioned unit instead of a folder of loose YAML: something you can install, upgrade, roll back, publish and depend on, by name and version.

3 • How does it work?

Helm loads the folder (or archive), reads `Chart.yaml`, merges `values.yaml` with your values, renders `templates/`, and sends the result to Kubernetes. `helm package` turns the folder into `NAME-VERSION.tgz`, named from `Chart.yaml`.

4 • What problem does it solve?

Copies. One chart serves every environment and every team that installs it.

5 • How do I use it?

Start with `helm create NAME` (conventional skeleton) or an empty folder with `Chart.yaml` + `templates/` ([challenge 1](#)). Adapt templates to the application, put every setting that varies into `values.yaml`.

6 · What command should I run?

```
helm create my-app           # skeleton
helm show chart charts/demo-app # read Chart.yaml of any chart (folder, .tgz, repo, oci://)
helm lint charts/demo-app     # validate
helm package charts/demo-app   # → demo-app-1.0.0.tgz
helm package charts/demo-app --version 1.0.1 --app-version 1.1.0 # set versions at package time (CI)
```

7 · What output should I expect?

`helm show chart` prints `Chart.yaml`; `helm package` prints `Successfully packaged chart and saved it to: ...`

8 · What can go wrong?

Problem	Symptom
Missing <code>name</code> / <code>version</code>	lint: <code>name is required</code> , <code>version ... is not a valid SemVer</code>
Unquoted <code>appVersion: 1.10</code>	becomes the number 1.1; lint error, wrong image tag (lab 03)
Contents changed, version not	two different packages called <code>demo-app-1.0.0.tgz</code> : users cannot trust versions
<code>type: library</code>	<code>helm install</code> refuses: library charts are not installable

9 · How do I troubleshoot it?

`helm lint` first; `helm show chart` to see what Helm actually parsed; `tar -tzf NAME.tgz` to see what a package contains (and whether `.helmignore` excluded the right files).

10 · Where is it used in real DevOps work?

Every application deployed with Helm has one: per service, or one shared chart for many similar services. Platform teams maintain charts for internal tools; vendors ship charts for their software.

Chart version vs application version

	Chart version (<code>version</code>)	Application version (<code>appVersion</code>)
Describes	the package: templates, defaults, options	the software deployed
Changes when	anything in the chart changes	the developers release
Format	SemVer, required	free text (quote it), usually SemVer
Seen in	<code>helm list</code> → <code>CHART demo-app-1.0.0</code> , package file names	<code>helm list</code> → <code>APP VERSION</code> , image tags
Example	<code>1.2.0</code> → <code>1.3.0</code> : new PDB option	<code>1.0.0</code> → <code>2.0.0</code> : new release of the app

They move independently, but a new `appVersion` changes the package's contents, so it also means a new chart version. Real-world example: chart `1.2.0` deploying application `2.0.0`.

Values

Hands-on: [lab 04](#), [lab 10](#), troubleshooting [05](#) and [12](#).

1 • What is it?

The settings a chart accepts. `values.yaml` holds every setting with a safe default and a comment; you override any of them per install. Templates read them as `.Values.<key>`.

```
replicaCount: 2
image:
  repository: ghcr.io/sufyanahmadkamboh/bookshop-frontend
  tag: "1.0.0"
service:
  type: ClusterIP
  port: 8080
```

2 • Why do we need it?

To separate **what** is deployed (templates, written once) from **how** it is configured (values, per environment).

3 • How does it work?

Helm merges every source into one tree, later sources overriding earlier ones:

```
chart's values.yaml < -f file1.yaml < -f file2.yaml < --set / --set-string / --set-json
```

Maps merge key by key; lists and scalars are replaced. `null` removes a key. A subchart receives the parent's values under its name (or alias); `global:` is visible to all charts.

4 • What problem does it solve?

Environment differences without copies: dev, staging and prod differ by a few lines of values.

5 • How do I use it?

Defaults in `values.yaml`; one file per environment with only the differences; `-f` in a fixed order from a script or pipeline; `--set` only for one-off, temporary overrides.

6 • What command should I run?

```
helm show values charts/demo-app          # the interface: every key and default
helm template demo charts/demo-app -f charts/demo-app/values-prod.yaml --set replicaCount=5
helm install demo charts/demo-app --dry-run=client --debug      # USER-SUPPLIED and COMPUTED VALUES
helm get values demo -n demo-dev [--all] [--revision N]         # what a release got
```

7 · What output should I expect?

`helm get values` prints `USER-SUPPLIED VALUES:` and only what you passed; `--all` adds the defaults.

8 · What can go wrong?

Mistake	Effect
Misspelled key (<code>replicacount</code>)	silently ignored (lab 04)
Forgotten <code>-f</code> on upgrade	release falls back to defaults (lab 08)
<code>-f values.yaml</code> passed last	resets every environment setting (lab 10)
Unquoted numbers/booleans	wrong types (<code>1.10</code> → <code>1.1</code> , <code>true</code> instead of <code>"true"</code>)
Coupled values	a value that "has no effect" because another disables it (12)
Secrets in values	in Git and in the release record (security)

9 · How do I troubleshoot it?

`helm get values` (did it arrive?) → `helm get manifest` / `helm template` (did it render?) → the template (which condition uses it?). A `values.schema.json` turns many of these mistakes into errors at lint time.

10 · Where is it used in real DevOps work?

Every deployment: environment configuration in reviewed values files, kept next to the chart or in a separate deployment repository read by CI/CD or GitOps tools.

Templates, helpers and labels

Hands-on: [lab 05](#), [lab 14](#), [examples/basic-chart](#).

1 • What is it?

Kubernetes manifests with Go-template placeholders, in `templates/`. **Helpers** are named templates (`{{ define "name" }}`) in files starting with `_` (usually `_helpers.tpl`), reused with `include` .

2 • Why do we need it?

To write a manifest once and let values fill in what differs; helpers to write shared pieces (names, labels) once.

3 • How does it work?

Helm renders each file as **text** with the values and built-in objects, then parses the text as YAML.

Object	Contains
<code>.Values</code>	the merged values
<code>.Release</code>	<code>Name</code> , <code>Namespace</code> , <code>Revision</code> , <code>IsInstall</code> , <code>IsUpgrade</code> , <code>Service</code>
<code>.Chart</code>	fields of <code>Chart.yaml</code> (<code>Name</code> , <code>Version</code> , <code>AppVersion</code>)
<code>.Capabilities</code>	the cluster's Kubernetes version and APIs
<code>.Template</code>	the current template's name and base path

The constructs this course uses: `{{ }}` , `if/else` , `range` , `with` , `define/include` , and the functions `default` , `quote` , `toYaml` , `nindent` , `required` , `tpl` , `sha256sum` , `lookup` , `fail` , `dict` , `printf` , `trunc` . [examples/basic-chart](#) uses each one, with a comment.

4 • What problem does it solve?

Duplication inside a chart (five services from one template in [bookshop](#)) and inconsistency (every object gets the same labels from one helper).

5 • How do I use it?

Keep templates close to plain Kubernetes YAML; add logic only where values must change the output; move repeated fragments into helpers; render after every change.

6 • What command should I run?

```
helm template demo charts/demo-app --show-only templates/deployment.yaml
helm template demo charts/demo-app --debug           # renders even invalid YAML, to see it
```

7 · What output should I expect?

Plain YAML, each document preceded by `# Source: <chart>/templates/<file>`.

8 · What can go wrong?

Error	Cause
unexpected EOF	an <code>if/range/with/define</code> without <code>end</code> (01)
YAML parse error	wrong indentation in the output; check <code>nindent</code>
valid YAML, wrong structure	wrong <code>nindent</code> that still parses (lab 05)
nil pointer evaluating	a parent key is missing (<code>.Values.a.b</code> with no <code>a</code>)
error calling include: template: no template "x"	helper name typo

9 · How do I troubleshoot it?

`helm lint`, `helm template --debug`, `--show-only` the file in question, and a strict schema check (`helm template ... | kubeconform -strict`).

10 · Where is it used in real DevOps work?

Avoiding duplicated Kubernetes YAML: every chart, every day. Reading templates is also how you review third-party charts.

Labels: why consistent metadata matters

The helper `bookshop.labels` gives every object `app.kubernetes.io/name`, `instance`, `version`, `managed-by`, `helm.sh/chart` and `environment`. That serves:

Need	How labels help
Kubernetes operations	Services and Deployments find Pods by <code>name</code> + <code>instance</code>
Debugging	<code>kubectl get all -l app.kubernetes.io/instance=shop</code> shows one release, all kinds
Monitoring	dashboards and alerts group by <code>name</code> , <code>version</code> , <code>environment</code>
Organisation	costs per <code>environment</code> , ownership audits via <code>managed-by</code>

Keep **selector** labels (name + instance) in their own helper and never change them: a Deployment's selector is immutable (troubleshooting 04).

Rendering and linting

Hands-on: [lab 06](#).

1 · What is it?

Rendering (`helm template`) turns a chart plus values into the plain YAML Helm would apply. **Linting** (`helm lint`) checks a chart for errors and bad practices.

2 · Why do we need it?

Don't trust the template just because Helm accepted it: render it. Seeing the exact output is the only way to know what will reach the cluster; lint catches broken charts in seconds, before a cluster does.

3 · How does it work?

Both run on your computer, without a cluster: defaults for `.Capabilities`, an empty `lookup`, no API validation. `helm install/upgrade --dry-run=server` does the same with the real cluster: real capabilities, real `lookup`, and the API server validates every object without creating it.

4 · What problem does it solve?

Late failures. Template syntax, values mistakes and malformed objects are found at review time, not at deploy time.

5 · How do I use it?

Lint and render for **every** environment's values in CI; render to a folder (`--output-dir`) or diff renders in code review; `--dry-run=server` just before applying.

6 · What command should I run?

```
helm lint charts/demo-app -f charts/demo-app/values-prod.yaml [--strict]
helm template demo charts/demo-app -f charts/demo-app/values-prod.yaml [--show-only templates/x.yaml] [-
-output-dir DIR]
helm template demo charts/demo-app | kubeconform -strict -summary
helm install demo charts/demo-app --dry-run=server
```

7 · What output should I expect?

1 chart(s) linted, 0 chart(s) failed; [INFO] / [WARNING] / [ERROR] lines when not. `helm template` prints YAML.

8 · What can go wrong?

Lint stops at the first blocking error per stage: values files are parsed first, then `Chart.yaml`, then templates ([lab 06](#)). Some mistakes pass both lint and server dry-run (wrong indentation that still parses): only a strict schema check or a careful read catches them.

9 · How do I troubleshoot it?

Fix one error, lint again; `--debug` to see invalid output; `--show-only` to focus on one file.

10 · Where is it used in real DevOps work?

The "render check" job of every chart's CI pipeline ([this repository's](#)), and in reviews: "show me the rendered diff".

Releases: install, status, history, uninstall

Hands-on: [lab 07](#).

1 · What is it?

A **release** is one installed instance of a chart: a name, a namespace, and a numbered history of **revisions**. Each revision stores the chart, the values and the rendered manifest, with a status (`deployed` , `superseded` , `failed` , `pending-install/upgrade/rollback` , `uninstalling`).

2 · Why do we need it?

To know what is deployed, with which configuration, since when, and to act on the application as a whole: upgrade it, roll it back, remove it.

3 · How does it work?

`helm install` renders the chart, applies the objects (Helm 4: server-side apply), and stores revision 1 as a Secret `sh.helm.release.v1.<name>.v1` in the release's namespace. Every later operation adds a revision. Helm keeps the last 10 revisions by default (`--history-max`).

4 · What problem does it solve?

`kubectl apply` keeps no record of what you applied as a whole. A release does.

5 · How do I use it?

`helm upgrade --install` with `--wait` from a pipeline; `helm list/status/history/get` to inspect; `helm uninstall` to remove.

6 · What command should I run?

Command	When
<code>helm install NAME CHART -n NS --create-namespace -f values.yaml --wait</code>	first deployment
<code>helm list [-A]</code>	what is installed, which chart/app versions
<code>helm status NAME</code>	current status and NOTES
<code>helm history NAME</code>	every revision and what happened
<code>helm get values\ manifest\ notes\ hooks\ metadata\ all NAME [--revision N]</code>	what exactly a revision contains
<code>helm uninstall NAME --wait</code>	remove the release's objects and history

7 · What output should I expect?

STATUS: deployed , REVISION: 1 , then the NOTES. `helm list` columns: NAME NAMESPACE REVISION UPDATED STATUS CHART APP VERSION .

8 · What can go wrong?

cannot reuse a name that is still in use (use `upgrade --install`); deployed without `--wait` while Pods fail (03); release Secrets deleted by hand (Helm forgets the release, objects stay); forbidden on Secrets (RBAC).

9 · How do I troubleshoot it?

`helm status` and `helm history` for Helm's view; `kubectl get ... -l app.kubernetes.io/instance=NAME` for the cluster's; `helm get manifest` | `kubectl diff --server-side --field-manager=helm -f -` for drift between them.

10 · Where is it used in real DevOps work?

Tracking application deployments: "which version runs in staging, deployed when, with which values?" is answered by `helm list` and `helm get values` .

What `helm uninstall` removes and leaves

Removed	Left behind
every object in the release's manifest	the namespace
all revision Secrets	PVCs from StatefulSet <code>volumeClaimTemplates</code>
	objects with <code>helm.sh/resource-policy: keep</code>
	hook objects (unless their delete policy removed them)
	CRDs from a chart's <code>crds/</code> folder

Verify after uninstalling: `helm list -n NS` and `kubectl get all,pvc,secret,configmap -n NS` .

Upgrades and rollbacks

Hands-on: [lab 08](#), [lab 09](#), troubleshooting [06](#) and [07](#).

1 • What is it?

`helm upgrade` applies a new chart version and/or new values to a release, as a new revision. `helm rollback` applies an earlier revision's manifest and values again, as a new revision.

2 • Why do we need it?

Deploying new application versions and configuration changes, and recovering quickly when one goes wrong.

3 • How does it work?

```
helm upgrade → render → apply changed/new objects, delete removed ones → Kubernetes rolling update  
                                         (new ReplicaSet, new Pods Ready, old Pods removed) → --wait until ready →  
revision N  
helm rollback REL M → apply revision M's manifest → revision N+1 "Rollback to M"
```

Helm manages the release; Kubernetes manages the workloads (`kubectl rollout status/history` shows the Deployment's side).

4 • What problem does it solve?

Safe change: every change is recorded, reviewable, and reversible as a whole.

5 • How do I use it?

Review first (`helm template ... | kubectl diff --server-side --field-manager=helm -f -`), always pass the complete values files, use `--wait` (and `--rollback-on-failure` in pipelines), verify with `helm test`.

6 • What command should I run?

```
helm upgrade demo charts/demo-app -n demo-dev -f charts/demo-app/values-dev.yaml --wait --timeout 3m  
kubectl rollout status deployment/demo-demo-app -n demo-dev  
helm history demo -n demo-dev  
helm rollback demo 2 -n demo-dev --wait  
helm upgrade ... --rollback-on-failure --timeout 60s      # automatic rollback (Helm 3: --atomic)
```

7 • What output should I expect?

Release "demo" has been upgraded. Happy Helming! ; Rollback was a success! Happy Helming! ; history lines
Upgrade complete , Rollback to N , or failed with the reason.

8 • What can go wrong?

Problem	Where
Forgotten values files → defaults	lab 08
Upgrade never ready (image, probes, OOM)	lab 09, 06
Successful but wrong upgrade	07
Partially applied failed upgrade (immutable field)	04
another operation ... is in progress	interrupted operation, lab 09
Server-side apply conflict	a field changed by another tool, 11

9 · How do I troubleshoot it?

`helm history` → `helm get values/manifest --revision N` diffs → `kubectl get pods`, `describe`, events of the new Pods.

10 · Where is it used in real DevOps work?

Upgrades: deploying a new application version. Rollbacks: recovering from failed deployments, the first move in an incident caused by a change. Remember that a rollback restores objects, not data.

Multiple environments

Hands-on: [lab 10](#), [Level 19](#).

1 · What is it?

One chart version deployed several times, once per environment, each release with its own values file and namespace.

2 · Why do we need it?

Changes must be tested in dev and staging with the same templates that will reach production; only configuration may differ.

3 · How does it work?

```
values.yaml (defaults) + values-dev.yaml    → release demo in demo-dev
                        + values-staging.yaml → release demo in demo-staging
                        + values-prod.yaml    → release demo in demo-prod
```

4 · What problem does it solve?

Environment drift and copy-paste (Level 1's three folders of 460 lines each).

5 · How do I use it?

Defaults that are safe for a laptop; one file per environment with only the differences; the same command shape for every environment (`helm upgrade --install demo CHART -n demo-$ENV -f values-$ENV.yaml --wait`); promotion = the same chart version with the next environment's file.

6 · What command should I run?

```
helm upgrade --install demo charts/demo-app -n demo-staging --create-namespace -f charts/demo-app/values-staging.yaml --wait
diff <(helm get values demo -n demo-staging) <(helm get values demo -n demo-prod)
kubectl get deploy -A -l app.kubernetes.io/name=demo-app -L environment
```

7 · What output should I expect?

The same chart version in every `helm list` row; values diffs limited to the intended differences.

8 · What can go wrong?

Wrong file for the environment (05), wrong file order ([lab 10](#)), environments drifting because changes were made with `--set` in one of them.

9 · How do I troubleshoot it?

`helm get values` per environment and diff them; the `environment` label on every object.

10 · Where is it used in real DevOps work?

Environment-specific configuration: replicas, resources, hosts, ingress/TLS, feature flags, autoscaling, persistence sizes, and **references** to secrets (never the secrets).

Typical difference	dev	staging	prod
Replicas / autoscaling	1	2	HPA
Image tag	default	default	pinned
Resources	small	small	measured
Host / TLS	dev host	staging host	public host + TLS
Persistence	1Gi	2Gi	5Gi+
Secrets	generated	existing Secret	external secret manager

Repositories and OCI registries

Hands-on: [lab 11](#), the Traefik install in [lab 00](#).

1 · What is it?

How charts are distributed. A **chart repository** is an HTTP server with an `index.yaml` and `.tgz` packages. An **OCI registry** (GHCR, ECR, Docker Hub, Harbor...) stores charts as OCI artifacts, like container images.

2 · Why do repositories exist?

So that a chart is consumed by **name and version**, not by copying a folder: the publisher releases versions, users pin and upgrade them.

3 · How do charts get distributed?

```
author: helm package → .tgz → upload + index.yaml (HTTP repo)    or    helm push oci://registry/path
user:   helm repo add + helm repo update → helm install repo/chart --version X
        helm install oci://registry/path/chart --version X        (no repo add needed)
```

4 · What problem does it solve?

Sharing charts across teams and organisations, with versions, and installing well-known software (ingress controllers, monitoring, databases) from its maintainers.

5 · How do I use it?

Add only trusted repositories; `helm search repo`; read `helm show chart/values/readme` and render before installing; always `--version`; keep your values in a file.

6 · What command should I run?

```
helm repo add podinfo https://stefanprodan.github.io/podinfo && helm repo update podinfo
helm search repo podinfo --versions
helm search hub podinfo                                # Artifact Hub
helm show values podinfo/podinfo --version 6.15.0
helm install catalog podinfo/podinfo --version 6.15.0 -f my-values.yaml
helm pull oci://ghcr.io/stefanprodan/charts/podinfo --version 6.15.0
helm registry login ghcr.io && helm push demo-app-1.0.0.tgz oci://ghcr.io/OWNER/charts
```

7 · What output should I expect?

"podinfo" has been added to your repositories; search tables with `CHART VERSION` and `APP VERSION`; `Pushed:` and `Digest:` for OCI.

8 · What can go wrong?

chart "x" matching 1.2.3 not found ... (try 'helm repo update') : stale index or a version that does not exist; 401/403 from a registry (login, or the artifact is private / does not exist); unpinned installs that change over time.

9 · How do I troubleshoot it?

helm repo update, helm search repo NAME --versions, helm show chart REF --version X; for OCI, helm registry login and the registry's UI.

10 · Where is it used in real DevOps work?

Platform teams installing third-party infrastructure; companies publishing internal charts to a private OCI registry; CI pushing a new chart version on every release ([this repository](#) pushes to GHCR).

Chart dependencies

Hands-on: [lab 12](#), [troubleshooting 10](#).

1 • What is it?

A parent chart that includes other charts (subcharts), declared in its `Chart.yaml` :

```
dependencies:
- name: postgres
  version: 0.1.1
  repository: "file:///./postgres"    # or https://..., oci://...
  condition: postgres.enabled        # optional switch
  alias: db                          # optional local name
```

```
Parent chart (bookshop)
├─ Application (templates/: 5 services, config, secret, ingress, hook, test)
└─ Dependency (charts/postgres-0.1.1.tgz)
    └─ Database (StatefulSet + volume + Service)
```

2 • Why do we need it?

To install an application together with the components it needs, as one release.

3 • How does it work?

`helm dependency update` resolves versions, downloads packages into `charts/`, writes `Chart.lock`; `helm dependency build` restores exactly what the lock says. At install, subcharts render with their own templates and the values the parent puts under their name; all objects belong to the parent's release.

4 • What problem does it solve?

Packaging applications with required components, without copying the components' templates.

5 • How do I use it?

Pin versions; commit `Chart.lock`, not the `.tgz`; run `helm dependency build` in CI; switch optional components with `condition`; configure them through the parent's values.

6 • What command should I run?

```
helm dependency list charts/bookshop
helm dependency update charts/bookshop    # after changing dependencies in Chart.yaml
helm dependency build charts/bookshop     # fresh clone / CI
```

7 • What output should I expect?

`helm dependency list`: STATUS ok for each dependency (missing before build/update).

8 · What can go wrong?

found in `Chart.yaml`, but missing in `charts/` directory ; `Chart.lock` is out of sync ; can't get a valid version ; subchart values under the wrong key (they must be under the dependency's name or alias).

9 · How do I troubleshoot it?

`helm dependency list` ; compare `Chart.yaml` , `Chart.lock` and what the source offers; `helm template` and look for `helm.sh/chart: <subchart>-<version>` labels.

10 · Where is it used in real DevOps work?

Umbrella charts for a whole product, applications bundling a cache or database for dev/CI, shared library charts.

When to use a dependency, and when not

Dependency	Separate release
Exists only for this app, same lifecycle	Shared by many apps (ingress, monitoring)
Dev, CI, demos: one command for everything	Production databases: managed service or operator, own backups and upgrades
Always installed and removed together	Owned by another team, upgraded on its own schedule

Hooks, tests and NOTES

Hands-on: [lab 13](#), [lab 14](#), [troubleshooting 11](#).

1 · What is it?

- A **hook** is an object (usually a Job) annotated `helm.sh/hook: <event>` that Helm runs at a lifecycle moment: `pre-install`, `post-install`, `pre-upgrade`, `post-upgrade`, `pre-delete`, `post-delete`, `pre-rollback`, `post-rollback`, `test`.
- A **test** is a hook of type `test`: a Pod that `helm test` runs on demand; exit 0 = pass.
- **NOTES.txt** is a template printed after install/upgrade and by `helm status`.

2 · Why do we need it?

Hooks: work that must happen at a precise moment of a release (a migration before the new version, a report after it). Tests: a one-command check that a release works. NOTES: tell the person who installed what to do next.

3 · How does it work?

Helm creates hook objects at their event, ordered by `helm.sh/hook-weight`, **waits** for them, and fails the operation if one fails. Delete policies (`before-hook-creation`, `hook-succeeded`, `hook-failed`) decide when hook objects are removed. Hook objects are not part of the release's manifest.

4 · What problem does it solve?

Ordering and verification that plain manifests cannot express.

5 · How do I use it?

Prefer normal resources; use hooks only for "at this moment" work; always set a delete policy; keep hook Jobs fast and idempotent. Give every chart a test; run it after each deployment and on a schedule.

6 · What command should I run?

```
helm get hooks shop -n bookshop-dev
helm test shop -n bookshop-dev --logs [--filter name=shop-test-books]
helm get notes shop -n bookshop-dev
helm install ... --no-hooks           # skip hooks (rarely a good idea)
```

7 · What output should I expect?

`helm test`: `TEST SUITE`, `Phase: Succeeded`, and with `--logs` the test Pod's output.

8 · What can go wrong?

A failing `pre-upgrade` hook stops the upgrade before any change ([lab 13](#)); slow hooks slow every deployment; leftover hook Jobs; tests that pass while the app is broken (a test that ignores errors: [lab 14](#) shows `set -e` and command substitution); drift that only a test notices.

9 · How do I troubleshoot it?

The error names the hook object; `kubect1 logs job/<hook>` ; `helm history` shows the failed revision; `helm test --logs` shows which check failed.

10 · Where is it used in real DevOps work?

Database migrations (`pre-upgrade`), post-deployment smoke tests and notifications, pipeline gates (`helm test` after `helm upgrade`), validating chart deployments in CI.

ConfigMaps and Secrets in charts

Hands-on: [demo-app's ConfigMap](#), [bookshop's Secret](#), [Level 17 · Security](#), troubleshooting [08](#) and [09](#).

1 · What is it?

Templates that render configuration (ConfigMap) and credentials (Secret) from values, and wire them into Pods (`envFrom` , `env.valueFrom` , `volumes`).

2 · Why do we need it?

Configuration differs per environment and must reach the application; credentials must reach it without being written where they can leak.

3 · How does it work?

```
# ConfigMap from values
data:
  APP_ENV: {{ .Values.environment | quote }}
# Pod restarts when the configuration changes
annotations:
  checksum/config: {{ include (print $.Template.BasePath "/configmap.yaml") . | sha256sum }}
# Secret: generated once, kept across upgrades, or referenced by name
{{- $existing := lookup "v1" "Secret" .Release.Namespace $name }}
```

4 · What problem does it solve?

Configuration as reviewed values; credentials without values.

5 · How do I use it?

Non-secret settings: values → ConfigMap, with a checksum annotation. Secrets: **never** as values in Git. Use one of the safe patterns:

Pattern	Use it for
Generated in the cluster + <code>lookup</code> + <code>resource-policy: keep</code>	dev, CI, demos
<code>existingSecret: NAME</code> (the chart only references it)	staging, production
External secret manager → Secret (External Secrets Operator, Secrets Store CSI)	production
Encrypted in Git (SOPS / helm-secrets, Sealed Secrets)	GitOps setups

6 · What command should I run?

```
helm template shop charts/bookshop --set database.existingSecret=shop-db-prod | grep -c 'kind: Secret'
# 0
kubectl get secret shop-db -n bookshop-dev -o jsonpath='{.metadata.annotations}'
```

7 · What output should I expect?

With `existingSecret`, no Secret is rendered; the Pods reference the named one.

8 · What can go wrong?

Missing ConfigMap or Secret → `CreateContainerConfigError`; a Secret with the wrong value for an existing database; passwords in values files, `--set` lines, CI logs or the release record ([security](#)); a changed ConfigMap that does not restart Pods (no checksum).

9 · How do I troubleshoot it?

`kubectl describe pod` (names the missing object), `helm get values` (the reference), `kubectl get secret/configmap` (the object). Compare secret values by hash, never by printing them.

10 · Where is it used in real DevOps work?

Every application chart. Moving teams from "passwords in values files" to existing-Secret and secret-manager patterns is a common real-world task.

Why Helm does not make secrets secure

A Secret is base64, not encrypted; anyone with `get secrets` reads it. Helm stores every value in its release Secret, so a password passed with `--set` is readable from the release history. Templating changes where a secret is written, not who can read it.

Level 17 · Helm security

Time: 30 minutes. Needs the `shop` release in `bookshop-dev` (lab 12) and Traefik. The commands below are tested like the labs.

Installing a Helm chart creates Kubernetes resources. Chart security is Kubernetes security. A chart can create anything your credentials allow: cluster-wide permissions, privileged Pods, webhooks that see every request. Helm adds no sandbox. This lesson is a review routine you can apply to any chart, and the secret-handling rules that matter most.

1 · Don't trust a chart you have not read

Before installing a third-party chart, render it and look for the objects with the widest reach:

```
helm get manifest traefik --namespace traefik | grep -E '^kind:' | sort | uniq -c
```

```
1 kind: ClusterRole
1 kind: ClusterRoleBinding
1 kind: Deployment
1 kind: IngressClass
1 kind: Service
1 kind: ServiceAccount
```

Traefik, installed in lab 00, created a **ClusterRole** and a **ClusterRoleBinding**: permissions across the whole cluster. For an ingress controller that is expected (it watches Ingresses and Services in every namespace), but read what exactly it may do:

```
kubectl get clusterrole traefik-traefik -o jsonpath='{range .rules[*]}{.resources}{ "  "}{.verbs}{ "\n"}{end}'
```

```
["configmaps","nodes","services"] ["get","list","watch"]
["endpointslices"] ["list","watch"]
["pods"] ["get"]
["secrets"] ["get","list","watch"]
["ingressclasses","ingresses"] ["get","list","watch"]
["ingresses/status"] ["update"]
["namespaces"] ["list","watch"]
["ingressroutes","ingressroutetcps","ingressrouteudps","middlewares","middlewareetcps","servertransports","servertransporttcps","tlsoptions","tlsstores","traefikservices"] ["get","list","watch"]
```

It can read Secrets in every namespace (to load TLS certificates for Ingresses). Whoever controls that Pod can read your Secrets. That is the kind of fact to know, and accept deliberately, before installing.

The review checklist for any chart:

Look for	Why	How
ClusterRole , ClusterRoleBinding , *Webhook*Configuration , CustomResourceDefinition	cluster-wide power	helm template ... \ grep -E '^kind:' \ sort \ uniq -c
privileged: true , hostNetwork , hostPath , runAsUser: 0 , added capabilities	node-level access	helm template ... \ grep -nE 'privileged\ hostPath\ hostNetwork\ runAsUser'
Every image:	what you will run, from where, which tag	helm template ... \ grep 'image:' \ sort -u
Hooks and tests	extra Jobs/Pods with their own images (lab 11)	helm template ... \ grep -B3 'helm.sh/hook'
Dependencies	charts inside the chart, with their own maintainers	helm show chart , helm dependency list

2 · Images: pinned and known

```
helm get manifest shop --namespace bookshop-dev | grep 'image:' | sort -u
```

```
image: "ghcr.io/sufyanahmadkamboh/bookshop-frontend:1.0.0"
image: "ghcr.io/sufyanahmadkamboh/bookshop-go-status:1.0.0"
image: "ghcr.io/sufyanahmadkamboh/bookshop-java-api:1.0.0"
image: "ghcr.io/sufyanahmadkamboh/bookshop-node-api:1.0.0"
image: "ghcr.io/sufyanahmadkamboh/bookshop-python-api:1.0.0"
image: "postgres:18.6-alpine"
# 70 = the "postgres" user of the alpine image: the server runs as it from the start, never as
root
```

Every image has an explicit version; none is `latest` . Production goes further: pin by digest (`image@sha256:...`), scan images in CI (Trivy, Gype), and only allow trusted registries (an admission policy such as Kyverno or Gatekeeper).

3 · Least privilege for what the chart runs

The Bookshop chart runs every container as a non-root user, without privilege escalation, with all Linux capabilities dropped and the default seccomp profile, and does not mount a Kubernetes API token into Pods that never call the API:

```
kubectl get pods --namespace bookshop-dev -l app.kubernetes.io/instance=shop \
-o custom-
columns='POD:.metadata.name,NON_ROOT:.spec.securityContext.runAsNonRoot,USER:.spec.securityContext.runAs
User,TOKEN:.spec.automountServiceAccountToken'
```

POD	NON_ROOT	USER	TOKEN
shop-frontend-5d865884c8-bxc5k	true	101	false
shop-go-status-6dc7845b99-tpns7	true	65532	false
shop-java-api-7ff96666d4-7zmv8	true	10001	false
shop-node-api-fbb58c4c4-7hgsh	true	1000	false
shop-postgres-0	true	70	false
shop-python-api-9f4bcbc58-wr65p	true	10001	false
shop-report-g8nqj	true	1000	false
shop-test-services	true	65534	false

4 · Least privilege for who runs Helm

Helm has no permissions of its own: it uses the kubeconfig's identity. A CI pipeline that deploys one application should use a ServiceAccount limited to that application's namespace (a Role, not a ClusterRole), not a cluster-admin kubeconfig. Remember that Helm stores releases as Secrets: the deploying identity needs to read and write Secrets in the target namespace, and **anyone who can read Secrets there can read every release's values** (section 5).

5 · Secrets: why values files are the wrong place

Three facts, demonstrated:

Fact 1: values end up in Git. A values file is meant to be committed and reviewed. A password in it is a password in the repository history forever. This repository's values contain key names only:

```
grep -rniE 'password|token|secret' charts/*/values*.yaml
```

```
charts/bookshop/values-prod.yaml:2:# In a real production cluster also set database.existingSecret (and
postgres.auth.existingSecret) to a Secret
charts/bookshop/values.yaml:2:# Never put real secrets here: see database.existingSecret.
charts/bookshop/values.yaml:87: # Production: create the Secret yourself (or with an external secret
manager) and name it here.
charts/bookshop/values.yaml:88: # Empty: the chart creates "<release>-db" with a random password on
first install and KEEPS it on upgrades.
charts/bookshop/values.yaml:89: existingSecret: ""
charts/bookshop/values.yaml:94: enabled: true # false = use an external database
(set database.host and database.existingSecret)
charts/bookshop/values.yaml:98: existingSecret: "" # same as database.existingSecret
charts/demo-app/values.yaml:2:# (-f my-values.yaml, --set key=value). Values files hold configuration,
NEVER real secrets.
charts/demo-app/values.yaml:80: automount: false # the app never calls the Kubernetes API:
don't give it a token
charts/demo-app/values.yaml:94:imagePullSecrets: []
charts/postgres/values.yaml:10: # Name of an existing Secret with the key below. Empty = "<release
name>-db" (the parent chart creates it).
charts/postgres/values.yaml:11: existingSecret: ""
charts/postgres/values.yaml:12: passwordKey: DB_PASSWORD
```

Fact 2: --set is not a safe alternative. A value passed on the command line is stored in the release record, inside the cluster, readable by anyone who can read Secrets in that namespace:

```
helm install leaky examples/values-example --namespace sec-demo --create-namespace \
--set apiToken=example-not-a-real-token > /dev/null
helm get values leaky --namespace sec-demo
```

```
USER-SUPPLIED VALUES:
apiToken: example-not-a-real-token
```

And without Helm, with plain kubectl, from the release Secret (base64, base64 again, gzip: encoding, not encryption):

```
kubectl get secret sh.helm.release.v1.leaky.v1 --namespace sec-demo -o jsonpath='{.data.release}' \
| base64 -d | base64 -d | gunzip | grep -o '"apiToken": "[^"]*"' | head -1
```

```
"apiToken": "example-not-a-real-token"
```

It is also in your shell history and in CI logs if the command is printed.

Fact 3: templating a Secret does not make it secret. A chart's `secret.yaml` turns values into a Kubernetes Secret: base64-encoded, readable by anyone with `get secrets` permission. Helm adds no encryption anywhere.

```
helm uninstall leaky --namespace sec-demo --wait
kubectl delete namespace sec-demo --wait=false > /dev/null
```

Safe patterns

Pattern	How it works	In this repository
Generated in the cluster	the chart creates a random value on first install, <code>lookup</code> keeps it on upgrades; it never exists outside the cluster	<code>charts/bookshop</code> without <code>database.existingSecret</code> (dev, CI)
Existing Secret by name	the Secret is created outside the chart (by an admin, a pipeline, an operator); the values file only holds its name	<code>database.existingSecret: shop-db-prod</code>
External secret manager	an operator syncs from AWS Secrets Manager, Vault, Azure Key Vault... into a Secret: External Secrets Operator, Secrets Store CSI Driver	(the existing-Secret pattern is what the chart needs to support it)
Encrypted in Git	the file is committed encrypted; decrypted at deploy time: SOPS (often with the helm-secrets plugin), Sealed Secrets	—

The existing-Secret pattern in practice, with an example password typed by an operator (never committed):

```
kubectl create namespace sec-demo2 > /dev/null
kubectl create secret generic shop-db-external --namespace sec-demo2 --from-literal=DB_PASSWORD="$(head
-c 18 /dev/urandom | base64)"
helm template shop charts/bookshop --namespace sec-demo2 \
  --set database.existingSecret=shop-db-external --set postgres.auth.existingSecret=shop-db-external \
  | grep -E 'kind: Secret|name: shop-db' | sort | uniq -c
kubectl delete namespace sec-demo2 --wait=false > /dev/null
```

```
secret/shop-db-external created
      5                  name: shop-db-external
```

With `existingSecret` set, the chart renders **no** Secret at all; every Deployment and the database reference `shop-db-external` by name. The password never passes through Helm.

6 · Supply chain: trusted sources, pinned versions

- Add repositories from the project's official documentation, over HTTPS; prefer the maintainer's own registry.
- Pin chart versions (`--version` , `Chart.lock`); upgrade deliberately and read the chart's changelog.
- Verify what you can: signed charts (`helm install --verify` with provenance files), signed OCI artifacts (cosign), digests (`oci://...@sha256:...`).
- Review dependencies like the chart itself: `helm dependency list` , then the dependency's templates.
- Scan rendered manifests for misconfigurations in CI (Trivy config, Checkov, kube-score) alongside `helm lint` .

The 10 questions

- 1. What is it?** The practices that keep installing and running charts from becoming an attack path or a leak.
- 2. Why do we need it?** A chart is code that runs with your cluster credentials, written by someone else, and Helm stores every value you give it.
- 3. How does it work?** Review before install, restrict what runs (security contexts, RBAC) and who deploys (least-privilege identities), keep secrets out of values and out of Helm's records.
- 4. What problem does it solve?** Over-privileged third-party software, leaked credentials in Git or release history, unpinned images changing under you.
- 5. How do I use it?** Make the review checklist and the secret patterns a habit, and automate the checks in CI.
- 6. What command should I run?** `helm template ... | grep -E '^kind:', ... | grep image:` , `helm get manifest` , `helm get values` , `kubectl auth can-i` .
- 7. What output should I expect?** No surprises: cluster-wide objects you can justify, pinned images, no secret values in any values file or `helm get values` output.
- 8. What can go wrong?** Charts granting cluster-admin, images from unknown registries, passwords in `--set` lines and CI logs, `latest` tags, unreviewed upgrades of third-party charts.
- 9. How do I troubleshoot it?** Render and grep (above); `kubectl auth can-i --list --as=system:serviceaccount:NS:SA` to see what a chart's ServiceAccount may do; decode the release Secret to see what Helm stored.

10. Where is it used in real DevOps work? Platform teams reviewing every chart before it reaches a cluster, security reviews of release pipelines, secret-management migrations (values files → external secret managers).

See also: [ConfigMaps and Secrets](#), [best practices](#).

Next, Level 18: [lab 15 · Troubleshooting](#).

Reviewing changes before an upgrade (Helm diff)

Hands-on: [lab 08](#), [lab 16 step 13](#).

1 • What is it?

Seeing exactly what an upgrade will change in the cluster, before running it.

2 • Why do we need it?

A one-line values change can delete an Ingress, restart every Pod or change a selector. A diff of the **result** shows that; a diff of the values file does not.

3 • How does it work?

Render the new state, and let the API server compute what applying it would do, without changing anything:

```
helm template demo charts/demo-app -n demo-dev --skip-tests -f charts/demo-app/values-dev.yaml --set replicaCount=2 \
| kubectl diff --server-side --field-manager=helm -n demo-dev -f - \
| grep -E '^[+-]' | grep -vE 'generation:|meta\.helm\.sh/|^\s+annotations:$' || true
```

`--server-side --field-manager=helm` makes the preview behave like Helm 4's own apply; the `grep`s drop noise (Helm's ownership annotations, `generation`). The same pipeline with `helm get manifest REL` as input finds **drift** between the release and the cluster ([lab 14](#)).

4 • What problem does it solve?

Surprises during deployments; "LGTM" reviews of changes nobody actually saw.

5 • How do I use it?

In a pull request job (render and diff against the target environment) and before every manual upgrade.

6 • What command should I run?

The pipeline above, or the optional [helm-diff plugin](#):

```
helm plugin install https://github.com/databus23/helm-diff --version v3.15.15 --verify=false
helm diff upgrade demo charts/demo-app -n demo-dev -f charts/demo-app/values-dev.yaml
```

The plugin is popular and maintained; it is still a third-party binary that runs with your credentials. Pin its version, and note that Helm 4 requires `--verify=false` for plugins installed from Git (they carry no signature).

7 • What output should I expect?

- lines (live) and + lines (after the upgrade), per object.

8 · What can go wrong?

Secrets generated with `randAlphaNum` + `lookup` always show as changed in `helm template` (no `lookup` without a cluster); filter them; fields owned by other tools show conflicts (add `--force-conflicts` to the preview only); a client-side `kubectl diff` reports errors Helm would not hit (a changed Service port).

9 · How do I troubleshoot it?

Diff one object at a time (`--show-only`); compare `helm get manifest --revision N` between two revisions for after-the-fact reviews.

10 · Where is it used in real DevOps work?

Change review in CI/CD, GitOps tools (Argo CD and Flux show this diff in their UIs), change-management records.

Helm best practices

Each practice, with where this repository applies it.

Practice	Why	In this repository
Keep charts simple	logic in templates is code nobody tests	plain YAML with few conditions; complexity only where it removes duplication (bookshop's <code>range</code>)
Meaningful chart names	the name becomes resource names and labels	<code>demo-app</code> , <code>bookshop</code> , <code>postgres</code>
Semantic versioning	users must trust versions	chart <code>1.2.0</code> , <code>postgres 0.1.1</code> bumped when its security context changed
Pin application versions	<code>latest</code> changes under you	<code>appVersion</code> + explicit tags; prod pins <code>image.tag</code>
Avoid unnecessary <code>--set</code>	not reviewed, not repeatable, ends up in shell history and release records	environment files; overlay files for releases (<code>capstone</code>)
Values files for environment configuration	one chart, small differences	<code>values-dev/staging/prod.yaml</code>
No hard-coded environment settings in templates	templates are shared by all environments	hosts, replicas, sizes are values
Keep secrets out of Git	Git never forgets	generated Secret or <code>existingSecret</code> ; nothing secret in values (<code>security</code>)
Consistent labels	selection, monitoring, audits	<code>bookshop.labels</code> helper on every object
Reusable helpers	one place to change names and labels	<code>_helpers.tpl</code> in every chart
Validate with <code>helm lint</code>	catches broken charts in seconds	CI lints every chart with every values file
Render with <code>helm template</code>	see what you ship	CI renders + kubeconform
Test with <code>helm test</code>	a release is not done until it works	test Pods in <code>demo-app</code> and <code>bookshop</code>
Review changes before upgrades	surprises are expensive	server-side diff (<code>reviewing changes</code>)
Maintain chart documentation	a chart is an interface	chart READMEs with value tables (<code>helm show readme</code>)
Document required values	users need to know what to set	comments in <code>values.yaml</code> ; <code>values.schema.json</code> (<code>lab 04</code>)
Avoid overusing hooks	moving parts, leftovers, no rollback	one hook in <code>bookshop</code> , for a reason (<code>hooks</code>)
Keep dependencies intentional	each one is code you run	one dependency, switchable (<code>postgres.enabled</code>), pinned, locked

Practice	Why	In this repository
Always <code>--wait</code>	<code>deployed</code> should mean running	every install and upgrade in the labs
Same command shape everywhere	prevents wrong-file mistakes	<code>helm upgrade --install REL CHART -n NS -f values-\$ENV.yaml --wait</code>
Selector labels never change	Deployment selectors are immutable	separate <code>selectorLabels</code> helper (04)
Change via Helm only	drift breaks upgrades (Helm 4 conflicts) and tests	troubleshooting 11

Level 19 · A production-style chart

Time: 40 minutes. Needs the `shop` release in `bookshop-dev` (lab 12). The commands are tested like the labs.

`charts/bookshop` packages the whole Bookshop platform from Level 1: five services, a database, an Ingress, configuration and a password. This lesson reads it as a design: which Helm feature solves which problem of the 1,374 lines of raw YAML, and why.

From raw YAML to chart

Raw YAML (<code>environments/<env>/</code>)	Chart (<code>charts/bookshop/</code>)	Technique
<code>deployments.yaml</code> + <code>services.yaml</code> : 5 near-identical blocks each	<code>templates/services.yaml</code> : one block	<code>range</code> over <code>.Values.services</code>
per-environment replicas, resources, image tags	<code>values-<env>.yaml</code>	values files layered on <code>values.yaml</code>
<code>configmap.yaml</code> with hand-written service URLs	<code>templates/configmap.yaml</code>	URLs built from the release name and ports; <code>TARGETS</code> built with <code>range</code>
<code>secret.yaml</code> with a password in Git	<code>templates/secret.yaml</code>	generated + <code>lookup</code> , or <code>existingSecret</code>
<code>database.yaml</code> (StatefulSet + Service)	<code>charts/postgres</code> dependency	subchart with a <code>condition</code>
<code>ingress.yaml</code> with 5 paths	<code>templates/ingress.yaml</code>	<code>range</code> over <code>ingress.paths</code> , ports looked up with <code>index</code>
labels typed by hand, slightly different everywhere	<code>_helpers.tpl</code>	one <code>bookshop.labels</code> helper, <code>dict</code> to pass context
(missing)	<code>report-hook.yaml</code> , <code>tests/</code> , <code>NOTES.txt</code>	hook, test, notes
(missing)	<code>autoscaling.yaml</code>	HPA + PDBs, off by default, on in prod

```
wc -l environments/*.yaml | tail -1
cat charts/bookshop/templates/*.yaml charts/bookshop/templates/_helpers.tpl
charts/bookshop/templates/tests/*.yaml | wc -l
```

1374 total
351

The design decisions

One template for every service

`templates/services.yaml` ranges over the `services` map. Each entry carries only what differs between services: image name, port, user ID, probes, resources, whether it needs the database:

```
sed -n '/^ java-api:/,/^\$/p' charts/bookshop/values.yaml
```

```
java-api:
  enabled: true
  image: { name: bookshop-java-api, tag: "" }
  replicas: 1
  port: 8080
  runAsUser: 10001
  database: true
  probes: { liveness: /health, readiness: /ready, startup: /health } # the JVM starts slowly: a
startup probe
  resources:
    requests: { cpu: 250m, memory: 256Mi }
    limits: { cpu: "1", memory: 512Mi }
```

Adding a sixth service is an entry in `values.yaml`, not 60 lines of YAML. A service can be switched off per environment (`services.go-status.enabled: false`) and everything that refers to it (Ingress paths, `TARGETS`, the test) adapts, because those templates range over the same map.

Helpers with more than one argument

Templates inside `range` need both the loop variable and the root context (`$`). The helpers take a `dict` :

```
grep -nE '\$ctx :=|include "bookshop.labels"' charts/bookshop/templates/services.yaml
```

```
7:{{- $ctx := dict "root" $ "name" $name "version" $svc.image.tag }}
14:  {{- include "bookshop.labels" $ctx | nindent 4 }}
26:    {{- include "bookshop.labels" $ctx | nindent 8 }}
87:  {{- include "bookshop.labels" $ctx | nindent 4 }}
```

Values that are templates

The frontend's nginx needs fully qualified service URLs that contain the release name and namespace, which a values file cannot know. Values may contain template expressions; the chart renders them with `tpl` :

```
grep -n 'NODE_API_URL' charts/bookshop/values.yaml
helm template shop charts/bookshop --namespace bookshop-dev --show-only templates/services.yaml | grep -
A1 'NODE_API_URL'
```

```
25:    NODE_API_URL: "http://{{ .Release.Name }}-node-api.{{ .Release.Namespace
}}.svc.cluster.local:3000"
- name: NODE_API_URL
  value: "http://shop-node-api.bookshop-dev.svc.cluster.local:3000"
```

A password nobody types

`templates/secret.yaml`: random on first install, read back with `lookup` on upgrades, kept on uninstall (`helm.sh/resource-policy: keep`) because the database volume outlives the release. Production replaces it with `database.existingSecret` ([security](#)).

Config changes restart Pods

Every Deployment carries `checksum/config`, a hash of the rendered ConfigMap: change a setting and every service rolls; change nothing and nothing restarts.

Three environments, one chart version

Dev runs since lab 12. Add staging and production:

```
for env in staging prod; do
  helm upgrade --install shop charts/bookshop --namespace bookshop-$env --create-namespace \
    -f charts/bookshop/values-$env.yaml --wait --timeout 10m | grep -E '^(NAMESPACE|STATUS):'
done
```

What each environment consists of:

```
for env in dev staging prod; do
  echo "== $env: $(helm get manifest shop --namespace bookshop-$env | grep -E '^kind:' | sort | uniq -c
| awk '{printf "%s %s, ", $1, $3}')"
done
```

```
== dev: 1 ConfigMap, 5 Deployment, 1 Ingress, 1 Secret, 6 Service, 1 StatefulSet,
== staging: 1 ConfigMap, 5 Deployment, 1 Ingress, 1 Secret, 6 Service, 1 StatefulSet,
== prod: 1 ConfigMap, 5 Deployment, 1 HorizontalPodAutoscaler, 1 Ingress, 5 PodDisruptionBudget, 1
Secret, 6 Service, 1 StatefulSet,
```

```
kubectl get deployments --all-namespaces -l app.kubernetes.io/part-of=bookshop \
  -o custom-
columns='NAMESPACE:.metadata.namespace,NAME:.metadata.name,READY:.status.readyReplicas,IMAGE:.spec.templ
ate.spec.containers[0].image' \
  | grep -E 'NAMESPACE|node-api|java-api'
```

NAMESPACE	NAME	READY	IMAGE
bookshop-dev	shop-java-api	1	ghcr.io/sufyanahmadkamboh/bookshop-java-api:1.0.0
bookshop-dev	shop-node-api	1	ghcr.io/sufyanahmadkamboh/bookshop-node-api:1.0.0
bookshop-prod	shop-java-api	2	ghcr.io/sufyanahmadkamboh/bookshop-java-api:1.0.0
bookshop-prod	shop-node-api	2	ghcr.io/sufyanahmadkamboh/bookshop-node-api:1.0.0
bookshop-staging	shop-java-api	1	ghcr.io/sufyanahmadkamboh/bookshop-java-api:1.0.0
bookshop-staging	shop-node-api	2	ghcr.io/sufyanahmadkamboh/bookshop-node-api:1.0.0

```
for host in dev.bookshop staging.bookshop bookshop; do
  printf '%-18s ' "$host"; curl -s "http://$host.localhost:8080/config.js"; echo
done
```

```
dev.bookshop      window.APP_CONFIG = { ADMIN_URL: "", APP_ENV: "dev" };

staging.bookshop  window.APP_CONFIG = { ADMIN_URL: "", APP_ENV: "staging" };

bookshop          window.APP_CONFIG = { ADMIN_URL: "", APP_ENV: "prod" };
```

The tests of all three, the way a pipeline would gate a promotion:

```
for env in dev staging prod; do
  echo "$env: $(helm test shop --namespace bookshop-$env | grep Phase | awk '{print $2}')"
done
```

```
dev: Succeeded
staging: Succeeded
prod: Succeeded
```

Same chart version (`bookshop-1.2.0`) everywhere; the differences are 3, 9 and 32 lines of values.

Production extras

```
kubectl get hpa,pdb --namespace bookshop-prod
```

NAME	REFERENCE	TARGETS
MINPODS MAXPODS REPLICAS AGE		
horizontalpodautoscaler.autoscaling/shop-node-api	Deployment/shop-node-api	cpu: <unknown>/70% 2
6 2 36s		

NAME	MIN AVAILABLE	MAX UNAVAILABLE	ALLOWED DISRUPTIONS	AGE
poddisruptionbudget.policy/shop-frontend	1	N/A	1	36s
poddisruptionbudget.policy/shop-go-status	1	N/A	1	36s
poddisruptionbudget.policy/shop-java-api	1	N/A	1	36s
poddisruptionbudget.policy/shop-node-api	1	N/A	1	36s
poddisruptionbudget.policy/shop-python-api	1	N/A	1	36s

- The HPA owns node-api's replica count (2–6); the Deployment template omits `replicas` for it.
- A PodDisruptionBudget per service keeps at least one Pod running while a node is drained.
- Pinned image tags (`image.tag: "1.0.0"`): production never follows a moving default.
- A 5 GiB volume instead of 1 GiB.

What a real production values file would add (not possible on a laptop): `database.existingSecret` from a secret manager or `postgres.enabled: false` with a managed database, an Ingress host with TLS, resource numbers from measurements, topology spread across availability zones.

Versioning this chart

	chart version	app version	
today	1.2.0	1.0.0	
node-api 1.1.0 released	1.2.1	1.0.0	only node-api's tag changes in values:
services.node-api.image.tag			
all services 2.0.0	1.3.0	2.0.0	appVersion moves; templates gain what 2.0.0 needs
selector labels changed	2.0.0	2.0.0	breaking for existing releases: major chart
version			

`appVersion` is the default for every image; a single service can move ahead with its own tag. The capstone does exactly that with node-api 1.1.0.

Cleanup

⚠️ DESTRUCTIVE COMMAND · uninstalls staging and production and deletes their namespaces (and database volumes).

```
for env in staging prod; do
  helm uninstall shop --namespace bookshop-$env --wait
  kubectl delete namespace bookshop-$env --wait=false > /dev/null
done
```

shop in bookshop-dev stays for the capstone.

Next, Level 20: [lab 16](#) · [Capstone](#).

Helm cheat sheet

Helm 4.3. REL = release name, CHART = folder, .tgz, repo/chart or oci://..., NS = namespace.

Charts

Command	Does
helm create NAME	new chart skeleton
helm lint CHART [-f values.yaml] [--strict]	validate
helm template REL CHART [-f v.yaml] [--set k=v] [-n NS]	render to YAML, no cluster
helm template ... --show-only templates/x.yaml	one file
helm template ... --output-dir DIR	one file per object
helm template ... --debug	render even invalid YAML
helm package CHART [--version X] [--app-version Y] [-d DIR]	→ NAME-X.tgz
helm show chart\ values\ readme\ all CHART [--version X]	inspect any chart
helm pull CHART --version X [--untar]	download
helm dependency list\ update\ build CHART	subcharts, Chart.lock, charts/

Releases

Command	Does
helm install REL CHART -n NS --create-namespace -f v.yaml --wait --timeout 5m	install
helm upgrade --install REL CHART -n NS -f v.yaml --wait	install or upgrade (pipelines)
helm upgrade ... --rollback-on-failure	roll back automatically if it fails
helm upgrade ... --force-conflicts	take over fields changed by other tools (server-side apply)
helm install ... --dry-run=server	render with the cluster, validate, create nothing
helm list [-A] [--filter REGEX]	releases
helm status REL -n NS	status + NOTES
helm history REL -n NS	revisions
helm get values\ manifest\ notes\ hooks\ metadata\ all REL [--revision N] [--all]	what a revision contains
helm rollback REL N -n NS --wait	back to revision N (as a new revision)
helm test REL -n NS [--logs] [--filter name=POD]	run the chart's tests

Command	Does
<code>helm uninstall REL -n NS --wait</code>	remove objects and history

Repositories and registries

Command	Does
<code>helm repo add NAME URL</code> / <code>helm repo update [NAME]</code> / <code>helm repo list</code> / <code>helm repo remove NAME</code>	HTTP repositories
<code>helm search repo WORD [--versions]</code>	search added repos
<code>helm search hub WORD</code>	search Artifact Hub
<code>helm registry login HOST</code>	OCI login
<code>helm push NAME-X.tgz oci://HOST/PATH</code>	publish to OCI
<code>helm install REL oci://HOST/PATH/NAME --version X</code>	install from OCI

Values precedence

```
chart values.yaml < -f first.yaml < -f second.yaml < --set / --set-string / --set-json / --set-file
maps merge · lists replace · null deletes · --reuse-values keeps the last release's values (prefer passing files)
```

Templates

Construct	Example
value	<code>{{ .Values.replicaCount }}</code>
release / chart	<code>{{ .Release.Name }}</code> , <code>{{ .Release.Namespace }}</code> , <code>{{ .Chart.AppVersion }}</code>
default	<code>{{ .Values.image.tag \ default .Chart.AppVersion }}</code>
quote	<code>{{ .Values.environment \ quote }}</code>
if / else	<code>{{- if .Values.ingress.enabled }} ... {{- else }} ... {{- end }}</code>
range	<code>{{- range \$k, \$v := .Values.config }}{{ \$k }}: {{ \$v \ quote }}{{- end }}</code>
with	<code>{{- with .Values.nodeSelector }}nodeSelector: {{- toYaml . \ nindent 2 }}{{- end }}</code>
helper	<code>{{- define "x.labels" -}}...{{- end }} / {{- include "x.labels" . \ nindent 4 }}</code>
map/YAML block	<code>{{- toYaml .Values.resources \ nindent 12 }}</code>
required	<code>{{ required "image.repository is required" .Values.image.repository }}</code>

Construct	Example
values as templates	<code>{{ tpl .Values.url . }}</code>
restart on config change	<code>checksum/config: {{ include (print \$.Template.BasePath "/configmap.yaml") . \ sha256sum }}</code>
read from cluster	<code>{{ lookup "v1" "Secret" .Release.Namespace "name" }}</code>
stop with an error	<code>{{ fail "message" }}</code>
whitespace	<code>{{- trims before, -}}</code> trims after

Annotations

Annotation	Effect
<code>helm.sh/hook: pre-install,post-upgrade,test,...</code>	run at that moment
<code>helm.sh/hook-weight: "-5"</code>	order among hooks (low first)
<code>helm.sh/hook-delete-policy: before-hook-creation\ hook-succeeded\ hook-failed</code>	cleanup of hook objects
<code>helm.sh/resource-policy: keep</code>	<code>helm uninstall</code> leaves this object

Troubleshooting

```
troubleshooting/triage.sh REL NS # Helm + Kubernetes view in one screen
helm history REL -n NS # what happened
diff <(helm get values REL -n NS --revision 2) <(helm get values REL -n NS --revision 3)
helm get manifest REL -n NS | kubectl diff --server-side --field-manager=helm -n NS -f - # drift
kubectl get pods,endpointslices -n NS; kubectl describe pod POD -n NS; kubectl logs POD -n NS [--previous]
kubectl get events -n NS --sort-by=.lastTimestamp | tail
```

Message	Usually
unexpected EOF	missing <code>{{ end }}</code>
YAML parse error on ...	indentation (<code>nindent</code>)
cannot reuse a name that is still in use	use <code>helm upgrade --install</code>
another operation (install/upgrade/rollback) is in progress	interrupted operation → <code>helm rollback REL LAST_GOOD</code>
field is immutable	changed selector → new release / recreate
conflict with "kubectl-..."	field changed outside Helm → decide, then <code>--force-conflicts</code>
missing in charts/ directory	<code>helm dependency build</code>
context deadline exceeded with <code>--wait</code>	Pods never Ready → <code>kubectl describe pod</code>

Message	Usually
ImagePullBackOff	wrong image/tag, or no pull credentials
CreateContainerConfigError	missing ConfigMap/Secret/key

Troubleshooting labs

Level 18. Twelve broken Helm deployments, each broken on purpose and investigated step by step. Time: 10–15 minutes each. Run them in order or pick one; every scenario sets up its own starting point and cleans up after itself.

Don't just read the fix. Each lab follows the order you would use in production:

Problem → Symptoms → Investigation → Commands → Output Interpretation → Root Cause → Fix → Verification → Lesson Learned

#	Scenario	Where it breaks	Key commands
01	Template syntax error	before anything reaches the cluster	<code>helm lint</code> , <code>helm template --debug</code>
02	Wrong value	Pods start, never become Ready	<code>helm get values</code> , <code>helm get manifest</code> , <code>kubectl describe</code>
03	Incorrect image	Pods never start	<code>kubectl get pods</code> , <code>describe</code> , <code>logs</code>
04	Wrong Service selector	Pods fine, no traffic	<code>kubectl get endpointslices</code> , <code>--show-labels</code>
05	Wrong environment values	runs, with the wrong configuration	<code>helm get values</code> , <code>helm history</code>
06	Failed upgrade	upgrade fails, old version still serving	<code>helm status</code> , <code>kubectl get events</code>
07	Bad release revision	upgrade "succeeds", application broken	<code>helm history</code> , <code>helm get values --revision</code> , <code>helm rollback</code>
08	Missing ConfigMap	Pods fail after a restart	<code>kubectl describe pod</code> , <code>helm get manifest</code>
09	Secret problem	<code>CreateContainerConfigError</code>	<code>kubectl get secret</code> , <code>helm get values</code>
10	Dependency problem	chart cannot be built	<code>helm dependency list/update</code>
11	Failed Helm test	<code>helm test</code> fails, then <code>helm upgrade</code> conflicts	<code>helm test --logs</code> , server-side diff, <code>--force-conflicts</code>
12	Values conflict	a value "has no effect"	values precedence, <code>helm get manifest</code>

Scenarios 01–07, 11 and 12 use `demo-app` in the namespace `trouble`, on `http://trouble.localhost:8080`, with `values-trouble.yaml`. Scenarios 08 and 09 use the `shop` release in `bookshop-dev` (lab 12). Scenario 10 needs no cluster.

The investigation toolkit

Question	Helm	Kubernetes
What did I ask for?	<code>helm get values REL [--revision N]</code>	
What did Helm apply?	<code>helm get manifest REL [--revision N]</code>	
What happened when?	<code>helm history REL</code> , <code>helm status REL</code>	<code>kubectl get events --sort-by=.lastTimestamp</code>
What is running?		<code>kubectl get pods,deploy,svc,ingress -l app.kubernetes.io/instance=REL</code>
Why is a Pod unhappy?		<code>kubectl describe pod</code> , <code>kubectl logs [--previous]</code>
Does the live state match the release?	<code>helm get manifest REL \</code> <code>kubectl diff --server-side --field-manager=helm -f -</code>	
Would my change render?	<code>helm lint</code> , <code>helm template --debug</code>	<code>--dry-run=server</code>

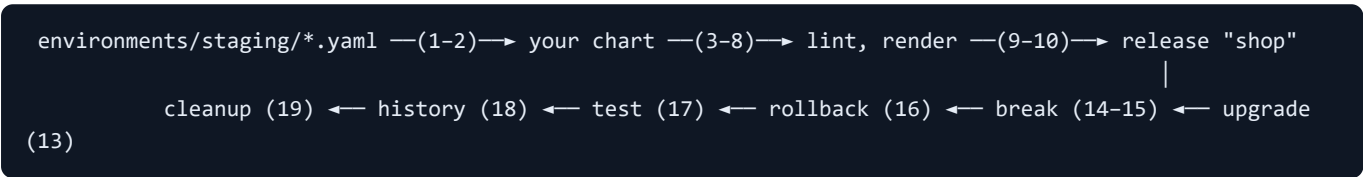
A general order that works: **render** (is the chart valid?) → **compare** (is what Helm applied what I meant?) → **observe** (what does Kubernetes say about the objects?) → **fix in values or chart, never only in the cluster** → **verify** (`helm test` , the application itself).

Lab 16 · Capstone: from raw YAML to a Helm-managed platform

Level 20. Time: 2–4 hours if you build the chart yourself, 45 minutes with the reference chart.

Objective

Take the multi-stack Bookshop from plain Kubernetes YAML to a Helm chart, and operate it through a full release lifecycle: install, verify, upgrade, a failed upgrade, troubleshooting, rollback, tests, history, cleanup.



Prerequisites

- Every lab so far, the [troubleshooting scenarios](#), and [Level 19](#).
- The cluster with Traefik. The `shop` release in `bookshop-dev` may still be running; the capstone works in `bookshop-staging`.

Task

Build a chart for the Bookshop (five services and PostgreSQL) that meets these requirements, then run the 19-step workflow below with it.

Requirement	Reference
Deployment, Service per service; one ConfigMap; one Secret; Ingress	lab 05
Persistent storage for the database	dependency, lab 12
Resource requests/limits, liveness/readiness (and startup) probes	lab 03
Consistent labels from helpers (app, version, environment, release)	lab 14
<code>values-dev.yaml</code> , <code>values-staging.yaml</code> , <code>values-prod.yaml</code> with only the differences	lab 10
A chart version and an application version	lab 03
<code>NOTES.txt</code> and a Helm test	lab 14
No password in any values file	Level 17

`charts/bookshop` is the **reference solution**. Build your own in `labs/work/` first if you want the full exercise; the commands below use the reference chart (replace `charts/bookshop` with your folder).

Commands

1 · Deploy manually using Kubernetes YAML

```
kubectl apply -f environments/staging/namespace.yaml > /dev/null
kubectl apply -f environments/staging/ > /dev/null
for d in frontend node-api java-api python-api go-status; do
  kubectl -n bookshop-staging rollout status deployment/$d --timeout=300s
done
```

```
curl -s http://staging.bookshop.localhost:8080/config.js; echo
kubectl get deployments --namespace bookshop-staging
```

```
window.APP_CONFIG = { ADMIN_URL: "", APP_ENV: "staging" };
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
frontend	2/2	2	2	27s
go-status	2/2	2	2	27s
java-api	2/2	2	2	27s
node-api	2/2	2	2	27s
python-api	2/2	2	2	27s

2 · Identify configuration duplication

```
echo "lines per environment: $(cat environments/staging/*.yaml | wc -l)"
echo "lines that differ from dev: $(diff -r environments/dev environments/staging | grep -c '^[<>]')"
grep -rn 'image:' environments/staging/deployments.yaml | sed 's/ *#.*//'
```

```
lines per environment: 458
lines that differ from dev: 74
22:      image: ghcr.io/sufyanahmadkamboh/bookshop-frontend:1.0.0
79:      image: ghcr.io/sufyanahmadkamboh/bookshop-node-api:1.0.0
130:     image: ghcr.io/sufyanahmadkamboh/bookshop-python-api:1.0.0
181:     image: ghcr.io/sufyanahmadkamboh/bookshop-go-status:1.0.0
224:     image: ghcr.io/sufyanahmadkamboh/bookshop-java-api:1.0.0
```

Write down what differs (namespace, replicas, `APP_ENV`, host, volume size, password) and what repeats (everything else). That list is your values design. Remove the manual deployment before Helm takes over the same names and host:

 **DESTRUCTIVE COMMAND** · deletes the namespace bookshop-staging and everything in it.

```
kubectl delete namespace bookshop-staging
```

3 · Create a Helm chart

```
mkdir -p labs/work/capstone
helm create labs/work/capstone/bookshop
```

Your starting point. Keep `_helpers.tpl`, `NOTES.txt`, `tests/`; replace the single Deployment with one template that ranges over the services.

4 · Convert YAML into templates · 5 · Move configuration into values

Compare one service, raw against rendered, to check your conversion keeps what matters:

```
echo "--- raw (environments/staging)"
sed -n '/name: node-api$/,/^---/p' environments/staging/deployments.yaml | grep -E
'replicas:image:containerPort:path:' | sed 's/ *#.*//'
echo "--- chart (charts/bookshop, staging values)"
helm template shop charts/bookshop -f charts/bookshop/values-staging.yaml --show-only
templates/services.yaml \
| sed -n '/name: shop-node-api$/,/^---/p' | grep -E 'replicas:image:containerPort:path:'
```

```
--- raw (environments/staging)
replicas: 2
  image: ghcr.io/sufyanahmadkamboh/bookshop-node-api:1.0.0
  containerPort: 3000
  httpGet: { path: /health, port: http }
  httpGet: { path: /ready, port: http }
--- chart (charts/bookshop, staging values)
replicas: 2
  image: "ghcr.io/sufyanahmadkamboh/bookshop-node-api:1.0.0"
  containerPort: 3000
  httpGet: { path: /health, port: http }
  httpGet: { path: /ready, port: http }
```

Same image, same port, same probes, same replica count, now from values.

6–8 · Values per environment

```
for env in dev staging prod; do echo "# values-$env.yaml"; grep -v '^#'
charts/bookshop/values-$env.yaml; done
```

```

# values-dev.yaml
environment: dev
ingress:
  host: dev.bookshop.localhost
# values-staging.yaml
environment: staging
services:
  node-api: { replicas: 2 }
  python-api: { replicas: 2 }
postgres:
  persistence:
    size: 2Gi
ingress:
  host: staging.bookshop.localhost
# values-prod.yaml
environment: prod
services:
  frontend:
    image: { tag: "1.0.0" }
    replicas: 2
  node-api:
    image: { tag: "1.0.0" }
    resources:
      requests: { cpu: 100m, memory: 128Mi }
      limits: { cpu: "1", memory: 512Mi }
  python-api:
    image: { tag: "1.0.0" }
    replicas: 2
  go-status:
    image: { tag: "1.0.0" }
    replicas: 2
  java-api:
    image: { tag: "1.0.0" }
    replicas: 2
report:
  image: { tag: "1.0.0" }
autoscaling:
  enabled: true
  minReplicas: 2
  maxReplicas: 6
podDisruptionBudget:
  enabled: true
postgres:
  persistence:
    size: 5Gi
ingress:
  host: bookshop.localhost

```

9 · Lint the chart · 10 · Render it

```

helm dependency build charts/bookshop > /dev/null
for env in dev staging prod; do
  helm lint charts/bookshop -f charts/bookshop/values-$env.yaml --quiet && echo "$env: lint ok"
  helm template shop charts/bookshop -f charts/bookshop/values-$env.yaml | grep -c '^kind:' | sed
"s/^/$env: objects rendered: /"
done
helm lint charts/bookshop -f charts/bookshop/values-staging.yaml | tail -1

```

```
dev: lint ok
dev: objects rendered: 17
staging: lint ok
staging: objects rendered: 17
prod: lint ok
prod: objects rendered: 23
1 chart(s) linted, 0 chart(s) failed
```

11 · Install the release

```
helm upgrade --install shop charts/bookshop --namespace bookshop-staging --create-namespace \
  -f charts/bookshop/values-staging.yaml --wait --timeout 10m
```

```
Release "shop" does not exist. Installing it now.
NAME: shop
LAST DEPLOYED: Mon Oct  5 01:22:30 2026
NAMESPACE: bookshop-staging
STATUS: deployed
REVISION: 1
DESCRIPTION: Install complete
NOTES:
Bookshop 1.0.0 (chart 1.2.0) · release "shop" · namespace "bookshop-staging" · environment staging ·
revision 1

Services:
  frontend      ghcr.io/sufyanahmadkamboh/bookshop-frontend:1.0.0 × 1
  go-status     ghcr.io/sufyanahmadkamboh/bookshop-go-status:1.0.0 × 1
  java-api      ghcr.io/sufyanahmadkamboh/bookshop-java-api:1.0.0 × 1
  ...
```

12 · Verify the application

```
kubectl get pods --namespace bookshop-staging
curl -s http://staging.bookshop.localhost:8080/config.js; echo
curl -s http://staging.bookshop.localhost:8080/api/users | head -c 100; echo
kubectl logs --namespace bookshop-staging job/shop-report | tail -1
```

NAME	READY	STATUS	RESTARTS	AGE
shop-frontend-5c9fb4cb45-sn77b	1/1	Running	0	26s
shop-go-status-59cc574b9d-zwlzh	1/1	Running	0	26s
shop-java-api-f7bcdffd7-ljbxn	1/1	Running	0	26s
shop-node-api-64f4cb4bcf-7kmmr	1/1	Running	0	26s
shop-node-api-64f4cb4bcf-rvr9c	1/1	Running	0	26s
shop-postgres-0	1/1	Running	0	26s
shop-python-api-5b48d98bf9-dtnxx	1/1	Running	0	26s
shop-python-api-5b48d98bf9-xx8c2	1/1	Running	0	26s
shop-report-rfsvk	0/1	Completed	0	3s

```
window.APP_CONFIG = { ADMIN_URL: "", APP_ENV: "staging" };

[{"id":1,"name":"Ada Lovelace","email":"ada@example.com","created_at":"2026-10-04T23:22:41.956Z"},{"
2026-10-04T23:22:55.849Z report-worker report #1 saved: users=3 books=5 reviews=0 services up=5/5
```

13 · Upgrade the application

The node-api team released **1.1.0**. Promote it to staging with a small overlay file, so the change is reviewable and repeatable (not a `--set` lost in a shell history):

```
cat > labs/work/capstone/node-api-1.1.0.yaml <<'EOF'
# Release overlay: node-api 1.1.0 (applied on top of the environment's values file)
services:
  node-api:
    image: { tag: "1.1.0" }
EOF
cat labs/work/capstone/node-api-1.1.0.yaml
```

Review the change before applying it (lab 08):

```
helm template shop charts/bookshop --namespace bookshop-staging --skip-tests \
-f charts/bookshop/values-staging.yaml -f labs/work/capstone/node-api-1.1.0.yaml \
| kubectl diff --server-side --field-manager=helm --namespace bookshop-staging -f - 2>/dev/null \
| grep -E '^[+]' | grep -vE 'generation:|meta\.helm\.sh/|^\s+annotations:$|DB_PASSWORD' || true
```

```
- app.kubernetes.io/version: 1.0.0
+ app.kubernetes.io/version: 1.1.0
-   app.kubernetes.io/version: 1.0.0
+   app.kubernetes.io/version: 1.1.0
-   image: ghcr.io/sufyanahmadkamboh/bookshop-node-api:1.0.0
+   image: ghcr.io/sufyanahmadkamboh/bookshop-node-api:1.1.0
- app.kubernetes.io/version: 1.0.0
+ app.kubernetes.io/version: 1.1.0
```

Only node-api's image and version label change (the Secret is filtered out: `helm template` renders a fresh random password every time, while the real upgrade keeps the existing one via `lookup`, lab 06).

```
helm upgrade shop charts/bookshop --namespace bookshop-staging \
-f charts/bookshop/values-staging.yaml -f labs/work/capstone/node-api-1.1.0.yaml \
--wait --timeout 5m | grep -E '^(STATUS|REVISION):'
```

```
kubectl rollout status deployment/shop-node-api --namespace bookshop-staging
helm test shop --namespace bookshop-staging --logs | grep node-api
```

```
deployment "shop-node-api" successfully rolled out
node-api: {"status":"ready","service":"node-api","version":"1.1.0"}
```

Expected Output

- Step 11: `STATUS: deployed`, revision 1, NOTES listing five services and the staging host.
- Step 13: revision 2, node-api reports `"version":"1.1.0"`.
- Step 16: revision 4, `Rollback to 2`, node-api still 1.1.0, java-api back to 1.0.0.

Explanation

The workflow you just ran is the one teams run every week: a values change (here the release overlay), reviewed as a diff, applied with `--wait`, verified by a test. Raw YAML gave you none of the safety rails: no

review diff tied to a release, no history, no rollback of the whole application, no test.

Break It

14 · Break the upgrade

Someone promotes java-api 2.0.0 before it was published:

```
helm upgrade shop charts/bookshop --namespace bookshop-staging \
  -f charts/bookshop/values-staging.yaml -f labs/work/capstone/node-api-1.1.0.yaml \
  --set services.java-api.image.tag=2.0.0 --wait --timeout 90s
```

```
...
Error: UPGRADE FAILED: resource Deployment/bookshop-staging/shop-java-api not ready. status: InProgress,
message: Pending termination: 1
context deadline exceeded
```

Troubleshoot It

15 · Troubleshoot

```
bash troubleshooting/triage.sh shop bookshop-staging | sed -n '/== helm history/,/== values/p;==
warning events/, $p'
```

```
== helm history (last 3)
REVISION      UPDATED              STATUS      CHART              APP VERSION DESCRIPTION
1             Mon Oct 5 01:22:30 2026  superseded  bookshop-1.2.0     1.0.0      Install complete
2             Mon Oct 5 01:22:59 2026  deployed    bookshop-1.2.0     1.0.0      Upgrade complete
3             Mon Oct 5 01:23:09 2026  failed      bookshop-1.2.0     1.0.0      Upgrade "shop" failed:
resource Deployment/bookshop-staging/shop-jav

== values supplied by the user
== warning events of the release's objects (last 8)
shop-node-api-64f4cb4bcf-rvr9c      Unhealthy    Readiness probe failed: Get
"http://10.244.1.189:3000/ready": context deadline exceeded (Client.Timeout exceeded
shop-node-api-64f4cb4bcf-7kmmr      Unhealthy    Readiness probe failed: HTTP probe failed with
statusCode: 503
shop-java-api-f7bcd added7-ljbxn      Unhealthy    Readiness probe failed: Get
"http://10.244.1.187:8080/ready": context deadline exceeded (Client.Timeout exceeded
shop-node-api-688cc5c85f-bvnbx      Unhealthy    Readiness probe failed: Get
"http://10.244.1.197:3000/ready": dial tcp 10.244.1.197:3000: connect: connection ref
shop-node-api-688cc5c85f-6djkb      Unhealthy    Readiness probe failed: Get
"http://10.244.1.198:3000/ready": dial tcp 10.244.1.198:3000: connect: connection ref
shop-java-api-5dd4d8bfd7-54n4k      Failed       Error: ImagePullBackOff
shop-java-api-5dd4d8bfd7-54n4k      Failed       Error: ErrImagePull
shop-java-api-5dd4d8bfd7-54n4k      Failed       Failed to pull image "ghcr.io/sufyanahmadkamboh/bookshop-
java-api:2.0.0": rpc error: code = NotFound desc = faile
```

The history shows revision 3 failed ; the events show bookshop-java-api:2.0.0 cannot be pulled. The rest of the platform is untouched and the old java-api Pod keeps serving: the rolling update waits for a Ready new Pod.

16 · Roll back

Revision 2 (node-api 1.1.0, everything else 1.0.0) is the last good state:

```
helm rollback shop 2 --namespace bookshop-staging --wait --timeout 5m
```

Challenge

Promote the **same** tested state to production the way a release process would: package the chart as version **1.2.1** (its contents now carry node-api 1.1.0 for production as a reviewed values overlay), install it into **bookshop-prod** from the package, and gate it with the Helm test.

Solution

▼ Open the solution

```
helm package charts/bookshop --version 1.2.1 --destination labs/work/capstone > /dev/null
helm upgrade --install shop labs/work/capstone/bookshop-1.2.1.tgz --namespace bookshop-prod --
create-namespace \
  -f charts/bookshop/values-prod.yaml -f labs/work/capstone/node-api-1.1.0.yaml --wait --timeout 10m
> /dev/null
helm test shop --namespace bookshop-prod
```

```
...
Last Started:   Mon Oct  5 01:24:59 2026
Last Completed: Mon Oct  5 01:25:00 2026
Phase:         Succeeded
```

`helm list` now shows `bookshop-1.2.1` in production and `bookshop-1.2.0` in staging: the chart version tells you exactly which package each environment runs. In a real setup, the package goes to a registry ([lab 11](#)) and production installs `oci://.../bookshop --version 1.2.1`.

Verification

17 · Run the Helm test

```
helm test shop --namespace bookshop-staging --logs | sed -n '/POD LOGS/, $p'
```

```
POD LOGS: shop-test-services (check)
frontend: {"status":"ok","service":"frontend","version":"1.0.0"}
go-status: {"service":"go-status","status":"ok","version":"1.0.0"}
java-api: {"status":"ready","service":"java-api","version":"1.0.0"}
node-api: {"status":"ready","service":"node-api","version":"1.1.0"}
python-api: {"status":"ready","service":"python-api","version":"1.0.0"}
frontend environment: staging
all checks passed
```

18 · Review the release history

```
helm history shop --namespace bookshop-staging | cut -c1-120
helm list --all-namespaces --filter '^shop$'
```

REVISION	UPDATED			STATUS	CHART	APP VERSION	DESCRIPTION
1	Mon Oct 5	01:22:30	2026	superseded	bookshop-1.2.0	1.0.0	Install complete
2	Mon Oct 5	01:22:59	2026	superseded	bookshop-1.2.0	1.0.0	Upgrade complete
3	Mon Oct 5	01:23:09	2026	failed	bookshop-1.2.0	1.0.0	Upgrade "shop" failed: resource Deployment/books
4	Mon Oct 5	01:24:41	2026	deployed	bookshop-1.2.0	1.0.0	Rollback to 2

NAME	NAMESPACE	REVISION	UPDATED	STATUS	CHART
shop	bookshop-dev	9	2026-10-05 01:12:17.5247962 +0200 CEST	deployed	bookshop-1.2.0 1.0.0
shop	bookshop-prod	1	2026-10-05 01:24:42.071881 +0200 CEST	deployed	bookshop-1.2.1 1.0.0
shop	bookshop-staging	4	2026-10-05 01:24:41.3077983 +0200 CEST	deployed	bookshop-1.2.0 1.0.0

Read it as the story of the week: installed (1), node-api 1.1.0 (2), a failed java-api promotion (3), rolled back to 2 (4). Nothing about this needed a meeting to reconstruct.

Cleanup

19 · Clean up

⚠️ DESTRUCTIVE COMMAND · uninstalls every shop release, deletes the namespaces, their database volumes and the kept password Secrets.

```
for ns in bookshop-dev bookshop-staging bookshop-prod; do
  helm uninstall shop --namespace $ns --wait 2>/dev/null
  kubectl delete namespace $ns --wait=false > /dev/null 2>&1
done
rm -rf labs/work/capstone
```

`helm uninstall` left the `shop-db` Secrets (`helm.sh/resource-policy: keep`) and the database volumes (StatefulSet claims): deleting the namespaces removes them. In a real environment, that is the moment to be sure: it deletes data.

```
helm list --all-namespaces
kubectl get namespaces
```


NAME	NAMESPACE	REVISION	UPDATED	STATUS	CHART	APP
VERSION						
traefik	traefik	1	2026-10-05 00:55:58.745498 +0200 CEST	deployed	traefik-41.6.1	
v3.7.13						
NAME		STATUS	AGE			
default		Active	29m			
kube-node-lease		Active	29m			
kube-public		Active	29m			
kube-system		Active	29m			
local-path-storage		Active	29m			
traefik		Active	29m			

You took an application from 1,374 lines of copied YAML to a chart with three short environment files, and operated it through a complete release lifecycle. Final step: [cleanup](#) of the cluster itself, and the [challenges](#).

Glossary

Every term used in this course, in plain words. The link points to where it is explained or used.

Term	Meaning
.helmignore	Files <code>helm</code> package leaves out of a chart archive, like <code>.gitignore</code> . lab 03
helpers.tpl	The conventional file for named templates. Files in <code>templates/</code> starting with <code></code> render nothing on their own. docs/04
alias (dependency)	A local name for a subchart: its objects and values use the alias (<code>catalog</code>) instead of the chart name. lab 12
apiVersion v2	The chart format of Helm 3 and 4, in <code>Chart.yaml</code> ; not a Kubernetes API version. docs/02
appVersion	The version of the application a chart deploys by default, usually the image tag. A string: quote it. docs/01
Artifact Hub	The public catalogue of Helm charts; <code>helm search hub</code> queries it. Listing is not an endorsement. lab 11
Base64	An encoding of bytes as text. Secret values and Helm's release records use it; it is not encryption. docs/13
Chart	A package of Kubernetes templates, default values and metadata: a folder or a <code>.tgz</code> . docs/02
Chart.lock	The exact dependency versions (and digest) a chart was built with. Committed; <code>helm dependency build</code> follows it. docs/10
Chart version	<code>version:</code> in <code>Chart.yaml</code> : the package's SemVer version. Changes whenever the chart's contents change. lab 03
Chart.yaml	A chart's identity: name, description, type, version, appVersion, dependencies. docs/02
checksum/config	A Pod annotation with a hash of the rendered ConfigMap, so configuration changes roll the Pods. lab 03
condition (dependency)	A value that switches a subchart on or off (<code>postgres.enabled</code>). lab 12
ConfigMap	A Kubernetes object holding non-secret configuration. docs/12
CreateContainerConfigError	A Pod status: a referenced ConfigMap, Secret or key is missing. troubleshooting 08
default (function)	Returns a fallback when a value is empty: <code>.Values.image.tag default .Chart.AppVersion</code> . lab 05
Dependency (subchart)	A chart included in another chart, declared in <code>Chart.yaml</code> , stored in <code>charts/</code> . docs/10
Drift	A difference between what Helm applied and what runs, caused by changes outside Helm. lab 14
--dry-run=server	Render with the real cluster, let the API server validate, create nothing. lab 06
existingSecret	A chart value naming a Secret created outside the chart; the chart only references it. docs/13

Term	Meaning
Field manager	The owner server-side apply records for each field. Helm 4 applies as <code>helm</code> ; other owners cause conflicts. troubleshooting 11
--force-conflicts	Lets Helm take over fields another manager owns. Use only when Helm's value should win. troubleshooting 11
global (values)	The one values key every chart in a tree (parent and subcharts) can read. docs/03
Helm	The package manager for Kubernetes: charts, templating, releases. docs/01
helm-diff	An optional plugin that shows what an upgrade would change. docs/14
Helper (named template)	A template defined with <code>define</code> and inserted with <code>include</code> . docs/04
Hook	An object Helm runs at a lifecycle moment (<code>pre-install</code> , <code>post-upgrade</code> , ...), annotated <code>helm.sh/hook</code> . lab 13
hook-delete-policy	When hook objects are deleted: <code>before-hook-creation</code> , <code>hook-succeeded</code> , <code>hook-failed</code> . lab 13
hook-weight	The order of hooks of the same event; lower first. lab 13
HPA	HorizontalPodAutoscaler: owns a Deployment's replica count between min and max. troubleshooting 12
Immutable field	A field Kubernetes refuses to change after creation, such as a Deployment's selector. troubleshooting 04
include	Inserts a named template's output, so it can be piped (<code>\ nindent 4</code>). lab 05
Ingress / IngressClass	HTTP routing rules / which controller (here Traefik) implements them. lab 00
kind	Kubernetes in Docker: each node is a container. The lab cluster. lab 00
kubeconform	A validator for Kubernetes YAML against the API schemas; <code>-strict</code> rejects unknown fields. lab 05
Library chart	<code>type: library</code> : only helpers for other charts; cannot be installed. docs/02
lint	<code>helm lint</code> : checks a chart for errors and bad practices. lab 06
lookup	A template function that reads an object from the cluster (empty with <code>helm template</code>). docs/16
Manifest (rendered)	The plain YAML a chart produced for a release; <code>helm get manifest</code> . lab 07
nindent	Adds a newline and indents every line by N spaces; keeps included YAML in place. lab 05
NOTES.txt	A template printed after install/upgrade and by <code>helm status</code> . lab 14
OCI registry	A container registry (GHCR, ECR...) that also stores charts; <code>oci://</code> references. lab 11
PDB	PodDisruptionBudget: keeps a minimum of Pods running during voluntary disruptions. docs/16
pending-upgrade	A release status written before an upgrade applies; left behind when the operation dies, it blocks further operations. lab 09

Term	Meaning
podinfo	A small, maintained demo application with a public chart, used here as a third-party chart. lab 11
Precedence (values)	<code>values.yaml</code> < <code>-f</code> files (later wins) < <code>--set</code> . lab 04
quote	Renders a value as a YAML string. lab 05
range	Repeats a block for each item of a list or map. lab 05
Release	One installed instance of a chart: name, namespace, revisions. docs/06
Release Secret	<code>sh.helm.release.v1.NAME.vN</code> : where Helm stores each revision (chart, values, manifest). lab 07
Repository (chart)	An HTTP server with <code>index.yaml</code> and chart archives. docs/09
required	Fails rendering with a message when a value is missing. docs/cheat-sheet
resource-policy: keep	Annotation that makes <code>helm uninstall</code> leave an object (the bookshop's password Secret). docs/16
Revision	One version of a release; every install, upgrade and rollback adds one. lab 08
Rollback	<code>helm rollback REL N</code> : re-applies revision N as a new revision. Restores objects, not data. lab 09
--rollback-on-failure	Rolls an upgrade back automatically if it fails (Helm 3: <code>--atomic</code>). lab 09
Selector labels	The labels a Service and Deployment select Pods with; never change them on a live release. docs/04
SemVer	MAJOR.MINOR.PATCH versioning; required for chart versions. docs/02
Server-side apply	Kubernetes computes the merge and tracks field owners; Helm 4's apply method. lab 07
--set	Command-line values, highest precedence; for one-off overrides. docs/03
Subchart	See Dependency.
Template	A manifest with Go-template placeholders, in <code>templates/</code> . docs/04
Test (Helm)	A Pod annotated <code>helm.sh/hook: test</code> , run by <code>helm test</code> . lab 14
tpl	Renders a string from values as a template (values containing <code>{{ .Release.Name }}</code>). docs/16
toYaml	Writes a value (a map) as YAML; usually followed by <code>nindent</code> . lab 05
Traefik	The ingress controller of the lab cluster, installed from its chart. lab 00
upgrade --install	Installs if the release is missing, upgrades otherwise: the pipeline command. lab 10
Values	The settings a chart accepts, defaults in <code>values.yaml</code> . docs/03
values.schema.json	A JSON Schema the merged values must satisfy, checked on lint, template, install and upgrade. lab 04
--wait	Wait until resources are ready (up to <code>--timeout</code>), so <code>deployed</code> means running. troubleshooting 03

Term	Meaning
with	Runs a block with <code>.</code> set to a value, only if the value is set. lab 05

Interview questions

25 questions that come up in interviews for DevOps, platform and cloud roles where Helm is used. Answer each one out loud first, then open the model answer. Each answer links to the lesson where you did it yourself: in an interview, "I did this in a lab, and here is what I saw" beats any memorised definition.

Fundamentals



1. What is Helm, and what problem does it solve?

The package manager for Kubernetes. It packages an application's manifests as a versioned chart, renders them with per-environment values, and installs them as a release with history. It solves duplication (one chart instead of a copy of the YAML per environment), drift, missing deployment history, whole-application rollback, and distribution of applications to other teams. ([Level 1](#), [docs/01](#))



1. Chart version vs app version: what is the difference?

`version` is the package (templates, defaults); `appVersion` is the software deployed by default (usually the image tag). A new template option changes the chart version only; a new application release changes `appVersion`, and the chart version too, because the package's contents changed. Published chart versions never change contents. ([lab 03](#))



1. What is a release, and where does Helm store it?

One installed instance of a chart, with a name, a namespace and numbered revisions. Each revision (chart, values, rendered manifest, status) is a Secret `sh.helm.release.v1.NAME.vN` in the release's namespace. No server component: anyone with access to the namespace sees the same releases. ([lab 07](#))



1. Explain values precedence.

The chart's `values.yaml`, then each `-f` file in order (later wins), then `--set` (highest). Maps merge key by key, lists are replaced, `null` deletes a key. Precedence decides a key's value, not whether a template uses it: with autoscaling on, `replicaCount` is ignored. ([lab 04](#), [troubleshooting 12](#))



1. helm template vs helm install --dry-run=server?

`helm template` renders locally with no cluster: default capabilities, empty `lookup`, no validation. `--dry-run=server` renders with the real cluster and lets the API server validate every object without creating any. Use the first in CI and reviews, the second just before applying. ([lab 06](#))

Templates



1. Why `nindent` and not just indentation in the template file?

Templates produce text. `include` returns a block without indentation; `nindent N` adds a newline and indents every line, so the block lands at the right YAML level. The wrong number can even produce valid YAML with a different structure (labels under `metadata` instead of `metadata.labels`) that lint and a server dry run accept. ([lab 05](#))



1. What are helpers for, and why separate selector labels from common labels?

Named templates written once (`define`) and reused (`include`): names and labels stay identical across objects. Selector labels go into their own helper because a Deployment's selector is immutable: adding a label there breaks upgrades and, partially applied, can break routing. ([troubleshooting 04](#))



1. How do you make Pods restart when only a ConfigMap changes?

Put a hash of the rendered ConfigMap into the Pod template annotations (`checksum/config: {{ include (print $.Template.BasePath "/configmap.yaml") . | sha256sum }}`). A config change changes the Pod template, which Kubernetes rolls out. ([lab 03](#))



1. How would you stop users from passing invalid values?

A `values.schema.json` : Helm validates the merged values on lint, template, install and upgrade (types, minimums, enums, required keys). Also `required` in templates, and `fail` for impossible combinations. ([lab 04](#))

Releases and operations



1. An upgrade succeeded but the app disappeared. What happened?

The likeliest cause is that the upgrade was run without the environment's values files: `helm upgrade` uses only the values given in that command, so the release fell back to defaults (here `ingress.enabled: false` ,

which deleted the Ingress). `helm get values` shows it immediately. Fix by upgrading with the full set of files; prevent it with a scripted command. ([lab 08](#))



1. Walk me through a rollback.

`helm history` to find the last good revision, optionally `helm get values/manifest --revision N` to confirm it, then `helm rollback REL N --wait`. It creates a new revision copying N; history is never rewritten. It restores objects and values, not data (migrations, volumes). `--rollback-on-failure` automates it for failing upgrades, but not for "successful" bad ones. ([lab 09](#))



1. "another operation (install/upgrade/rollback) is in progress": what do you do?

A previous operation died after writing a `pending-*` revision (a cancelled CI job, a closed laptop). Check that nothing is really running (CI, teammates), then `helm rollback REL LAST_GOOD`, which replaces the pending state. ([lab 09](#))



1. Why always use `--wait`?

Without it, Helm reports `deployed` as soon as the API server accepts the objects, even if no Pod can start. With `--wait` (and `--timeout`), Helm waits for readiness and marks the revision `failed` otherwise, which also makes `--rollback-on-failure` and pipeline gates meaningful. ([troubleshooting 03](#))



1. How do you review an upgrade before applying it?

Render the new state and diff it against the live objects with server-side apply semantics: `helm template ... | kubectl diff --server-side --field-manager=helm -f -`, or the `helm-diff` plugin. Review the result, not the values change. ([docs/14](#))



1. What does `helm uninstall` leave behind?

The namespace, StatefulSet volume claims, objects annotated `helm.sh/resource-policy: keep`, hook objects not removed by their delete policy, and CRDs from `crds/`. Verify the namespace afterwards, not just `helm list`. ([lab 07](#))

Environments, repositories, dependencies



1. How do you manage dev, staging and production with Helm?

One chart version for all; a values file per environment with only the differences; a namespace per environment; the same command shape everywhere (`helm upgrade --install REL CHART -n app-$ENV -f values-$ENV.yaml --wait`), promotion = the same chart version with the next file, gated by `helm test` . Secrets are referenced, never stored in values. (lab 10, Level 19)



1. How do you evaluate a third-party chart before installing it?

Who publishes it and is it maintained; `helm show chart/values/readme` ; render it with your values and look for cluster-wide objects (ClusterRoles, webhooks, CRDs), privileged settings and every image (including test Pods); pin the version (or digest); review its dependencies. (lab 11, security)



1. HTTP chart repository vs OCI registry?

An HTTP repository serves `index.yaml` plus `.tgz` files (`helm repo add/update , repo/chart`). An OCI registry stores charts as artifacts in a container registry (`helm push , oci://host/path/chart`), with registry authentication and digests, without an index. (lab 11)



1. dependency update vs dependency build, and what goes into Git?

`update` resolves `Chart.yaml` , downloads, and rewrites `Chart.lock` ; `build` downloads exactly what the lock says. Commit `Chart.yaml` and `Chart.lock` , not `charts/*.tgz` ; CI runs `build` . (lab 12)



1. When is a subchart the wrong choice?

When the component is shared by many applications (an ingress controller), has its own owner or lifecycle, or is a production database that needs its own backups and upgrades (often a managed service or an operator). A `condition` lets the same chart use a bundled database in dev and an external one in production. (docs/10)

Hooks, tests, security, troubleshooting



1. When would you use a Helm hook, and what are the risks?

For work that must happen at a precise moment of a release: a migration before the new version (`pre-upgrade`), a report or smoke check after it (`post-upgrade`). Risks: they block the operation, a rollback does

not undo their effects, their objects survive uninstall without a delete policy, and GitOps tools translate them differently. ([lab 13](#))



1. What makes a good Helm test?

It checks what a deployment can prove (Services route to Ready Pods, dependencies reachable, this release's configuration reached the app), runs as non-root, and really fails: `set -e`, no failures hidden inside `$( )`. Test it by breaking what it guards. ([lab 14](#))



1. How do you handle secrets in Helm charts?

Never in values files (Git) or `--set` (release record, shell history). Generate in the cluster with `lookup` and `resource-policy: keep` for dev; reference an existing Secret by name (`existingSecret`) for production, filled by an external secret manager (External Secrets Operator, CSI driver) or encrypted in Git (SOPS, Sealed Secrets). ([security](#))



1. helm upgrade fails with "conflict with kubectl-patch". Explain.

Helm 4 applies with server-side apply and records field ownership. Someone changed a field with kubectl, which now owns it; Helm refuses to overwrite another manager's change silently. Investigate the drift (`helm get manifest | kubectl diff --server-side --field-manager=helm -f -`), then use `--force-conflicts` if Helm's value should win. ([troubleshooting 11](#))



1. A Helm release is broken. Describe your troubleshooting method.

Triage the facts in a fixed order: `helm status`, `helm history`, `helm get values`, Pods, workloads, endpoints, warning events (this repository's `triage.sh`). Take the first anomaly as the hypothesis, confirm it with one command (`describe`, logs, a manifest diff), fix it in values or the chart, `helm upgrade --wait`, verify with `helm test` and the app, and repeat: a second fault can hide behind the first. ([lab 15](#))